Multimodal LLMs: Image, Video, and the Road to World Models

Dr. Che Guan

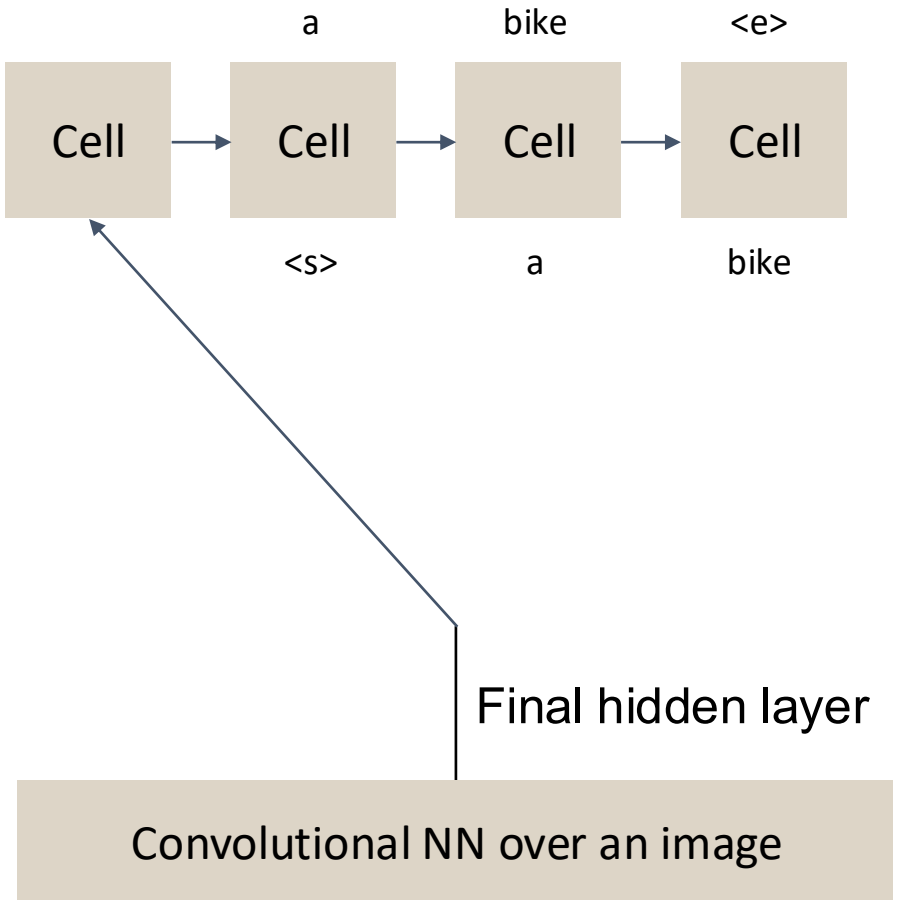
July 20, 2026

Part 1:

Image Captioning

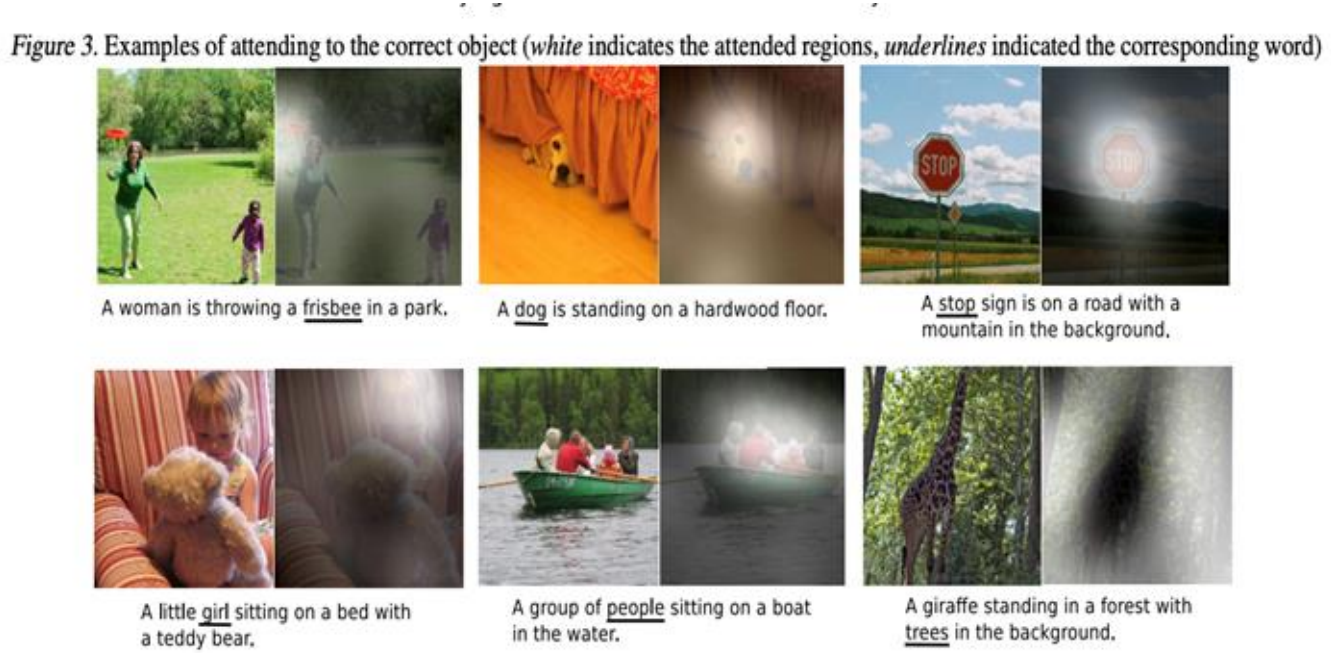
Approach 1: Show and Tell

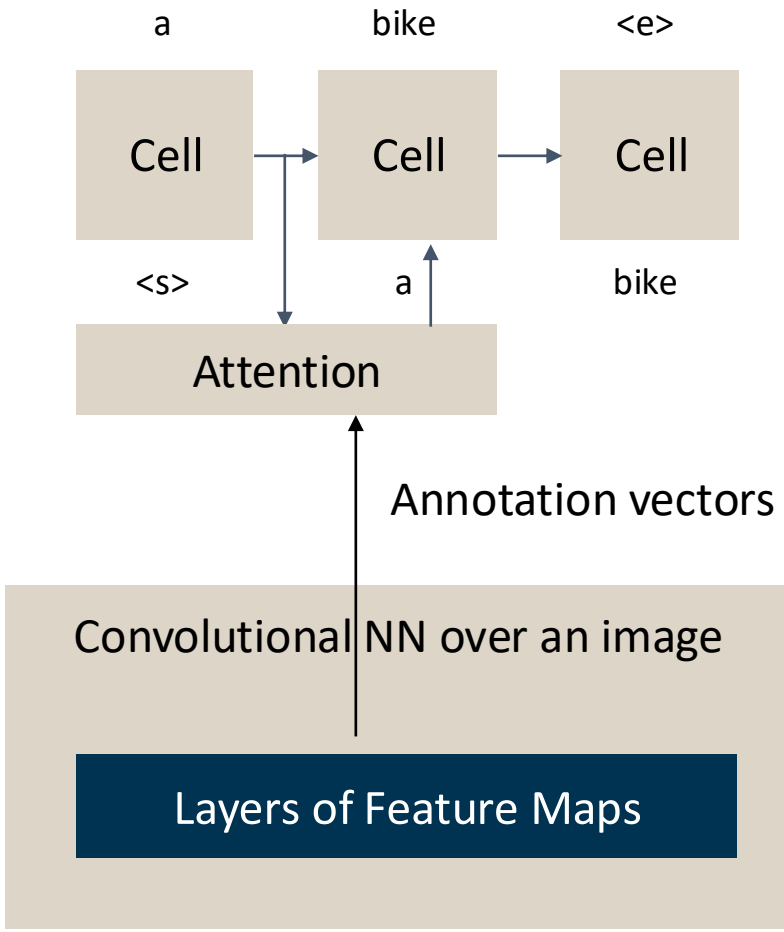
- Start with a strong ImageNet-trained image classifier
- Remove the classification head
- Add an LSTM: the CNN's features become its starting state
- Freeze the CNN; train the LSTM to predict the caption word by word
- Quiz:
  - What are two approaches for labeling images composed of multiple objects?
  - In either case, how would you decompose the modeling challenge?



Show and Tell: A Neural Image Caption Generator (https://arxiv.org/pdf/1411.4555.pdf)

Approach 2: Show, Attend and Tell

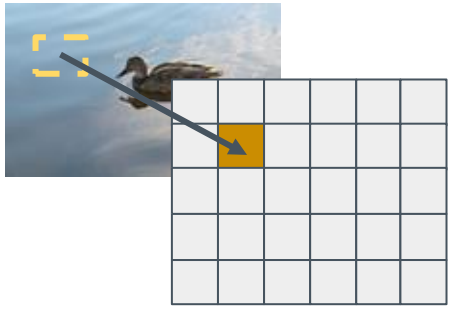




- Where do the annotation vectors come from?
- How do they enable the attention highlighting (e.g., in the figure above)?

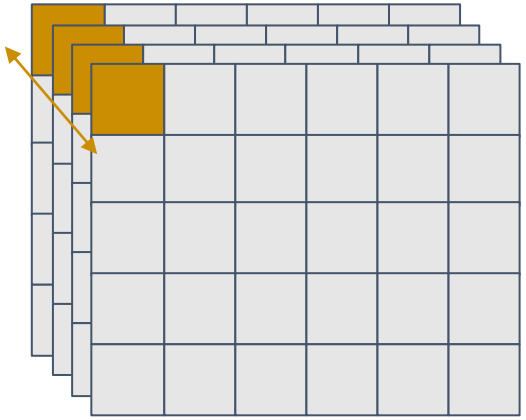
Show, Attend and Tell: Neural Image Caption Generation with Visual Attention (https://arxiv.org/pdf/1502.03044.pdf)

Annotation Vectors, Visualized



(multiplied by *n* kernels at this layer)

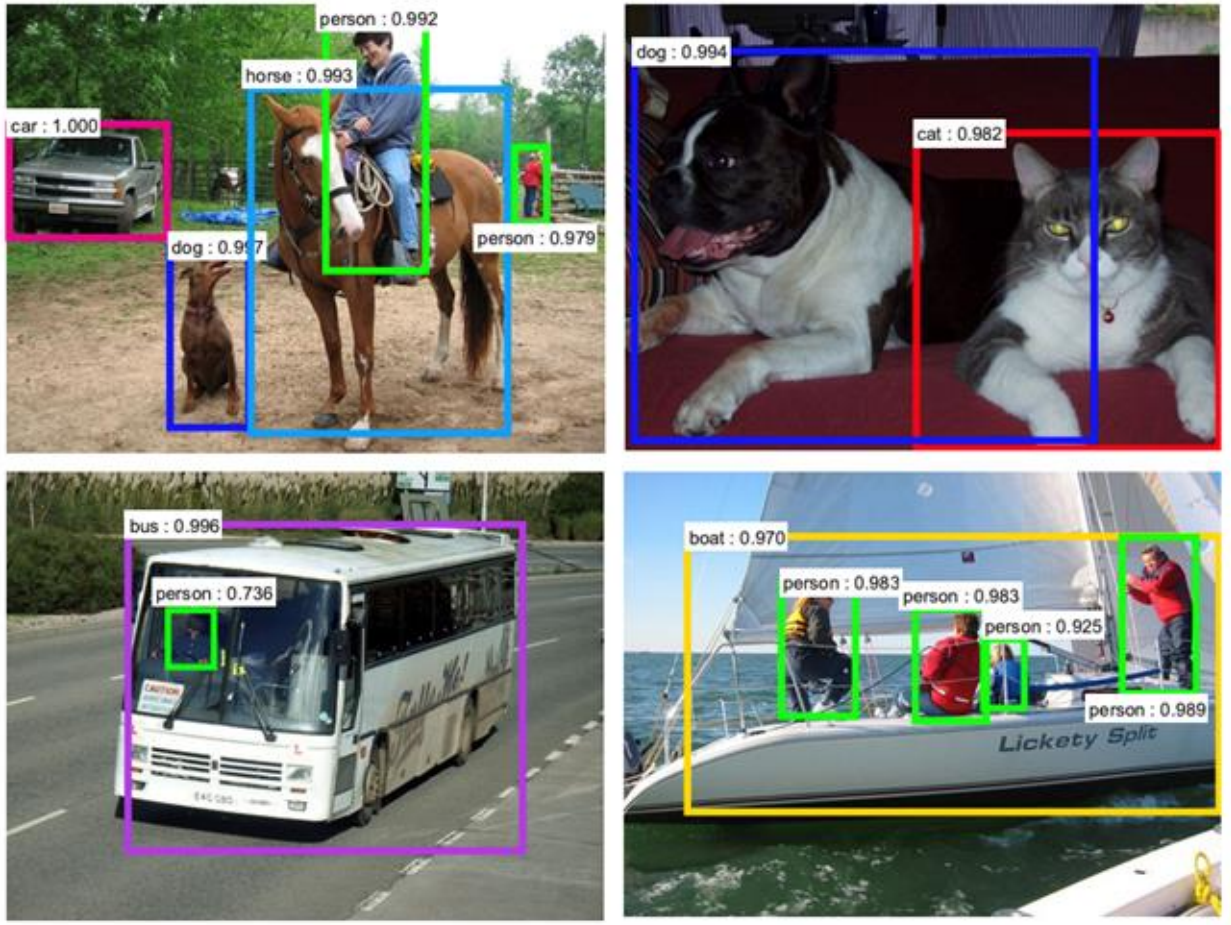
- Annotation vector corresponding to upper left corner of image



- Each layer of the CNN has multiple feature maps (one per kernel at that layer)
- Take the values at one position across all feature maps: that vector describes what appears at that spot in the image.
- We'll call each of these vectors an "annotation vector".

Images with Multiple Parts

- What are two approaches for labeling images composed of multiple objects?
- In either case, how would you decompose the modeling challenge?



Region Proposal Networks (RPN)

- RPNs propose regions of an image that each likely contain an object.
- Each region can then be separately classified
- The two steps usually share computation — feature maps line up spatially with the image so one backbone serves both

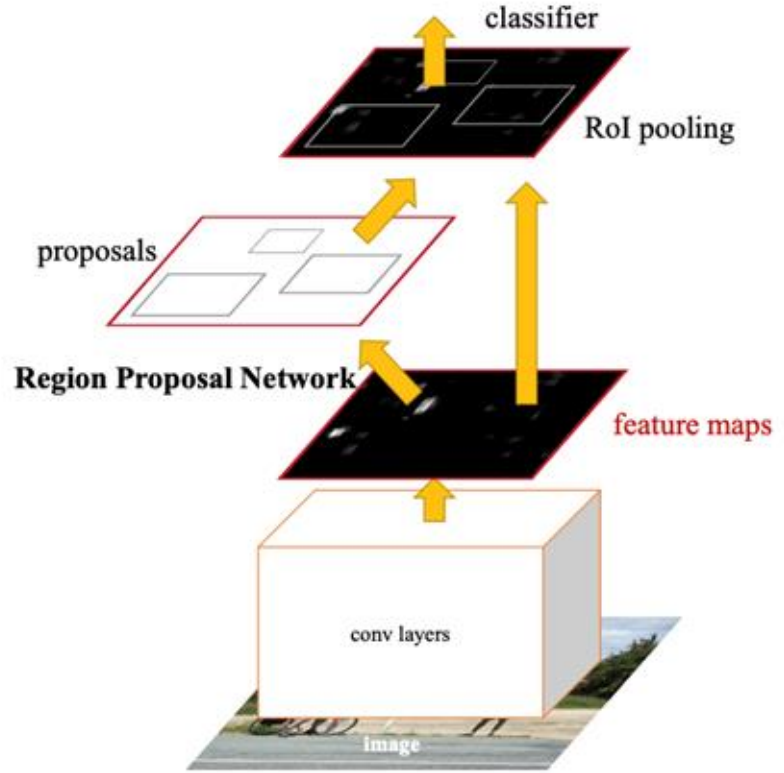


Figure 2: Faster R-CNN is a single, unified network for object detection. The RPN module serves as the 'attention' of this unified network.

Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks (https://arxiv.org/pdf/1506.01497.pdf)

Alignments for … Image Descriptions

- Regions don’t have to be labeled from a fixed list of classes.
- Each proposed region can be treated as its own small image and captioned in natural language

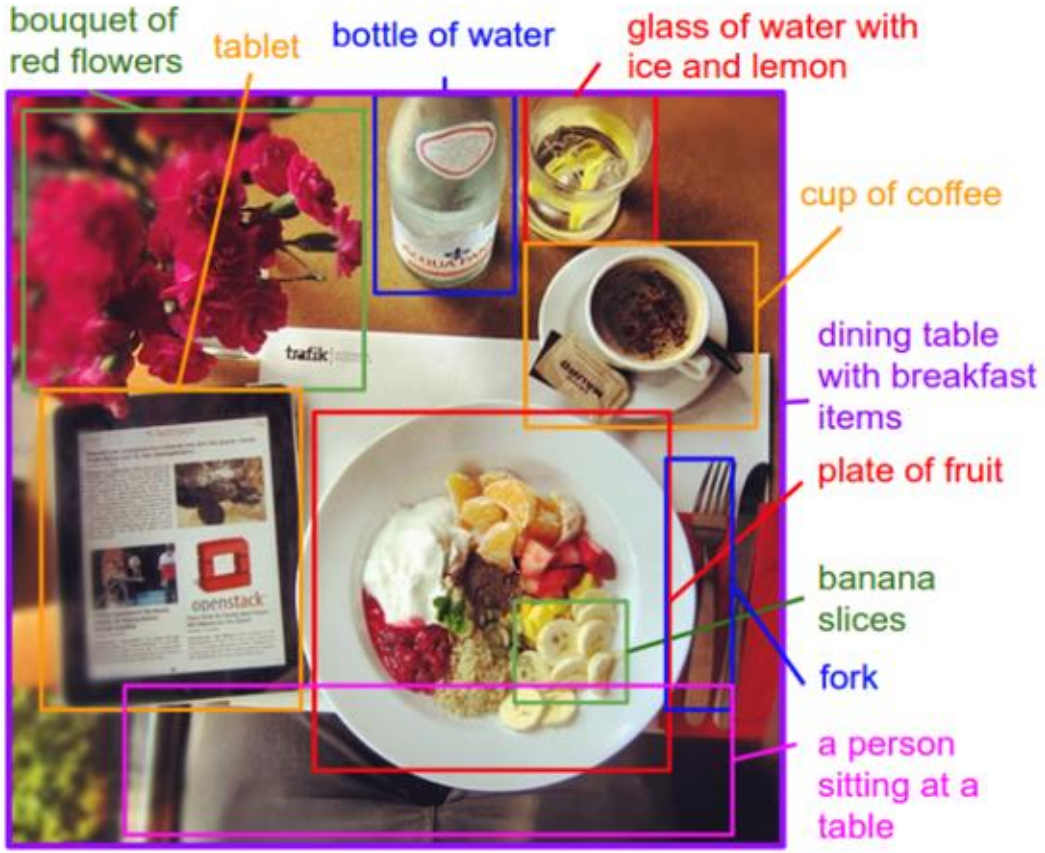


Figure 1. Motivation/Concept Figure: Our model treats language as a rich label space and generates descriptions of image regions.

Deep Visual-Semantic Alignments for Generating Image Descriptions (https://arxiv.org/pdf/1412.2306.pdf)

Using Regions as the Unit of Attention

- In “Show, Attend and Tell,” attention worked over a fixed grid of image locations.
- What if attention instead worked over the object regions proposed by an RPN?

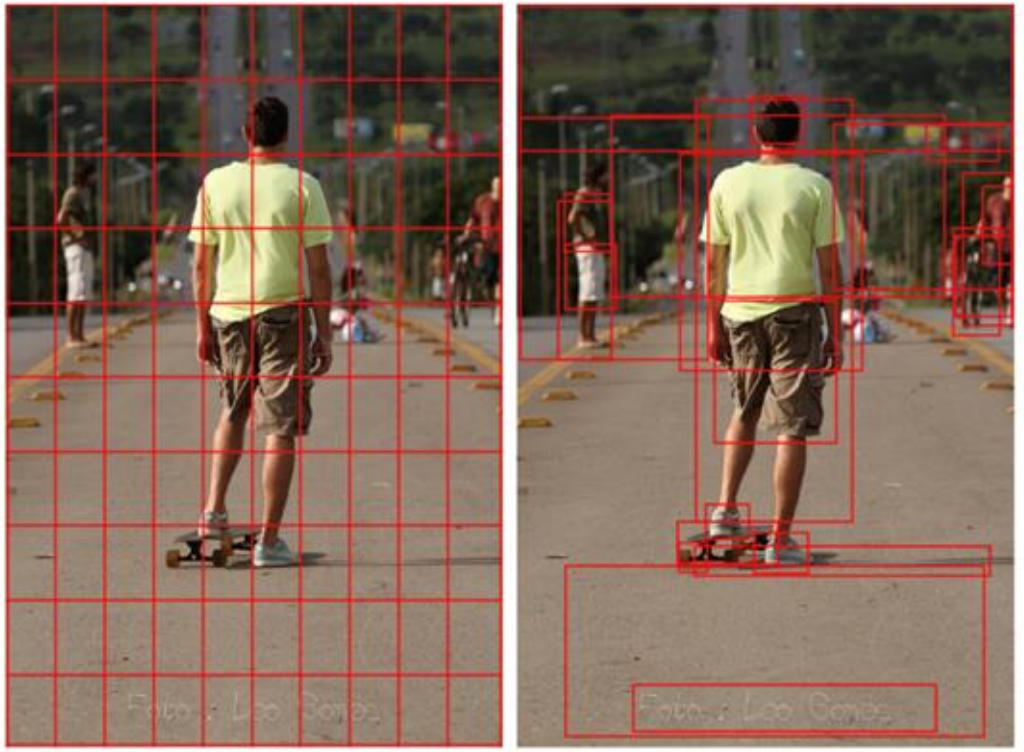
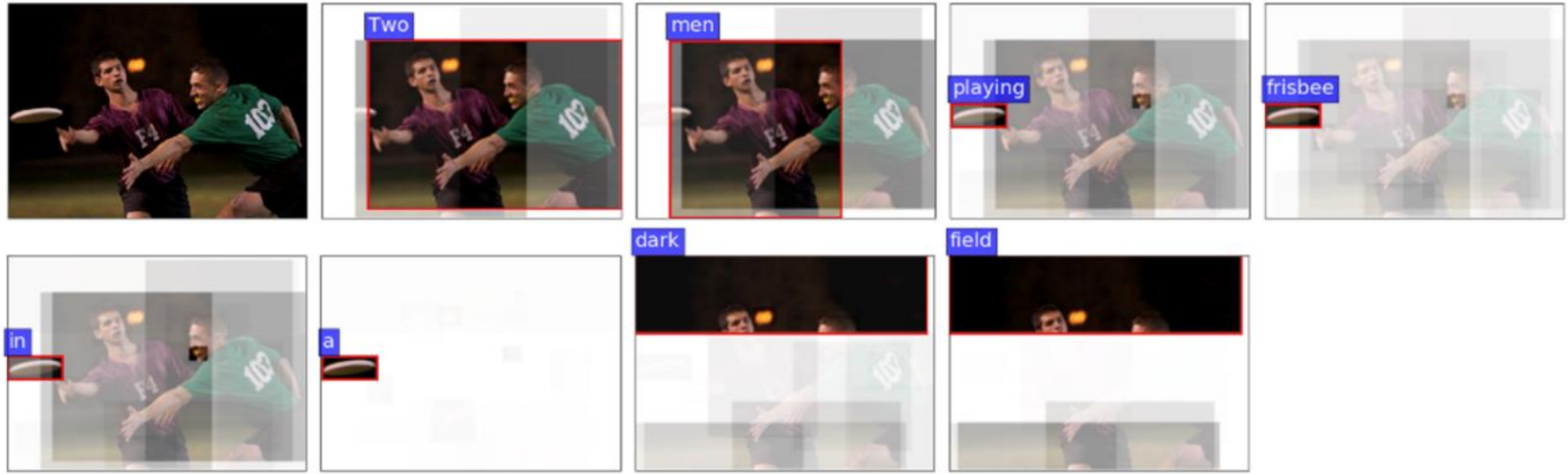


Figure 1. Typically, attention models operate on CNN features corresponding to a uniform grid of equally-sized image regions (left). Our approach enables attention to be calculated at the level of objects and other salient image regions (right).

Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering (https://arxiv.org/pdf/1707.07998.pdf)

Attention Follows the Words



Two men playing frisbee in a dark field.

Key idea: as the model writes each word of the caption, its attention (bright patch) moves to the matching part of the image.

Show, Attend and Tell: Neural Image Caption Generation with Visual Attention (https://arxiv.org/pdf/1502.03044.pdf)

Visual Question Answering

Input consists of two components:

- An image
- A question about the image: “What room are they in?”

What is output from these models?

Example (asset management): point a VQA model at a chart in an earnings deck and ask “Which quarter had the highest revenue?”

Visual Question Answering (VQA)

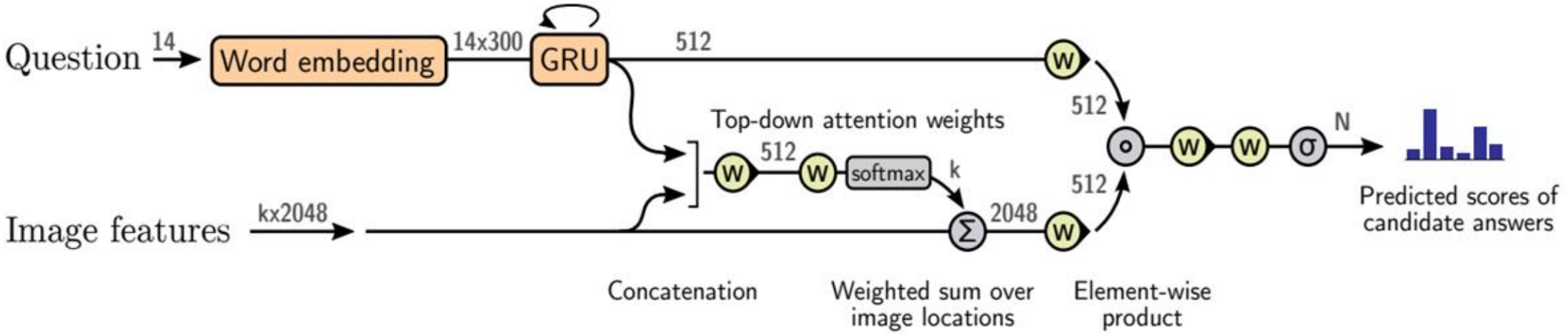


Figure 4. Overview of the proposed VQA model. A deep neural network implements a joint embedding of the question and image features $\{\boldsymbol{v}_1, ..., \boldsymbol{v}_k\}$. These features can be defined as the spatial output of a CNN, or following our approach, generated using bottom-up attention. Output is generated by a multi-label classifier operating over a fixed set of candidate answers. Gray numbers indicate the dimensions of the vector representations between layers. Yellow elements use learned parameters.

Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering (https://arxiv.org/pdf/1707.07998.pdf)

VQA Output

Question: What room are they in? Answer: kitchen

Figure 6. VQA example illustrating attention output. Given the question 'What room are they in?', the model focuses on the stove-top, generating the answer 'kitchen'.

Bottom-Up and Top-Down Attention for Image Captioning and Visual Question Answering (https://arxiv.org/pdf/1707.07998.pdf)

Part 2:

Image Generation Models

Image Generation

Two steps:

- Learn better image representations from huge amounts of loosely labeled image–text data from the web

- Use that image–text link to generate compelling images from text prompts

Fundamental contribution: Contrastive Language-Image Pre-Training (CLIP).

Learning Transferable Visual Models From Natural Language Supervision (https://arxiv.org/pdf/2103.00020.pdf)

CLIP: Contrastive Language-Image Pre-Training

- The lesson from LLMs: massive amounts of messy web data beat small, hand-labeled datasets
- Can we do the same for image representations?

Maybe!

- But existing image datasets are too small: MS-COCO and Visual Genome have ~100k images each; YFCC100M has only 15M usable captions

Learning Transferable Visual Models From Natural Language Supervision (https://arxiv.org/pdf/2103.00020.pdf)

CLIP, *Continued*

Dataset

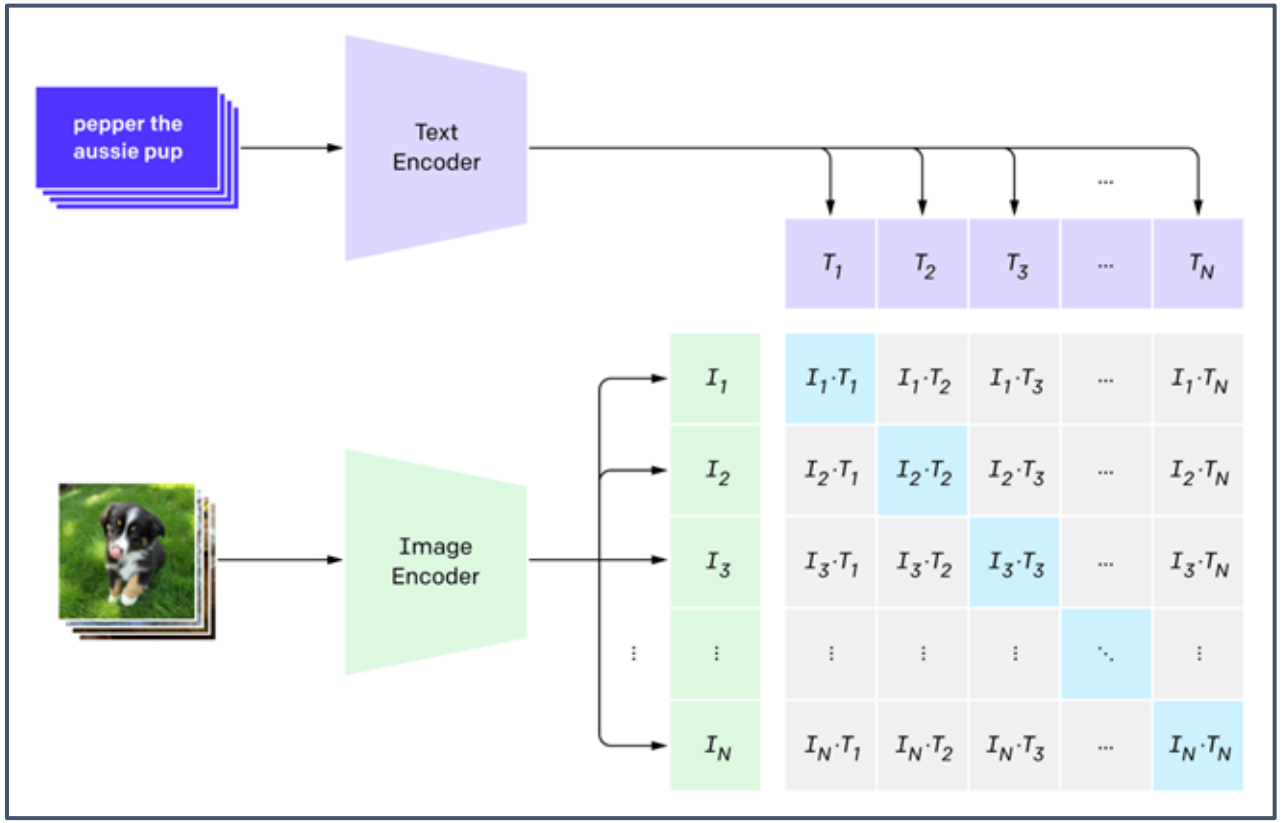
- Construct a 500k query list from:
  - popular terms on Wikipedia
  - bigrams with high mutual information
  - Titles of frequently searched Wikipedia articles
- Pull up to 20k (text, image) pairs from the results of each of those queries

Result: 400M (text, image) pairs — about as much text as GPT-2's whole training set

Learning Transferable Visual Models From Natural Language Supervision (https://arxiv.org/pdf/2103.00020.pdf)

CLIP, *Continued*

- **Model Setup**
  - With 400M pairs, training speed decides what is feasible
  - First try: predict each image's exact caption. Too slow — too much compute for too little gain.



  - Contrastive approach: a matching game.
    - In a batch of N (text, image) pairs, find the N true matches among $N \times N$ possible pairings.
    - The loss pulls true pairs together (high cosine similarity) and pushes the other $N^2 - N$ apart — about 4× more efficient than caption prediction.

Learning Transferable Visual Models From Natural Language Supervision (https://arxiv.org/pdf/2103.00020.pdf)

CLIP, *Continued*

Model Structure

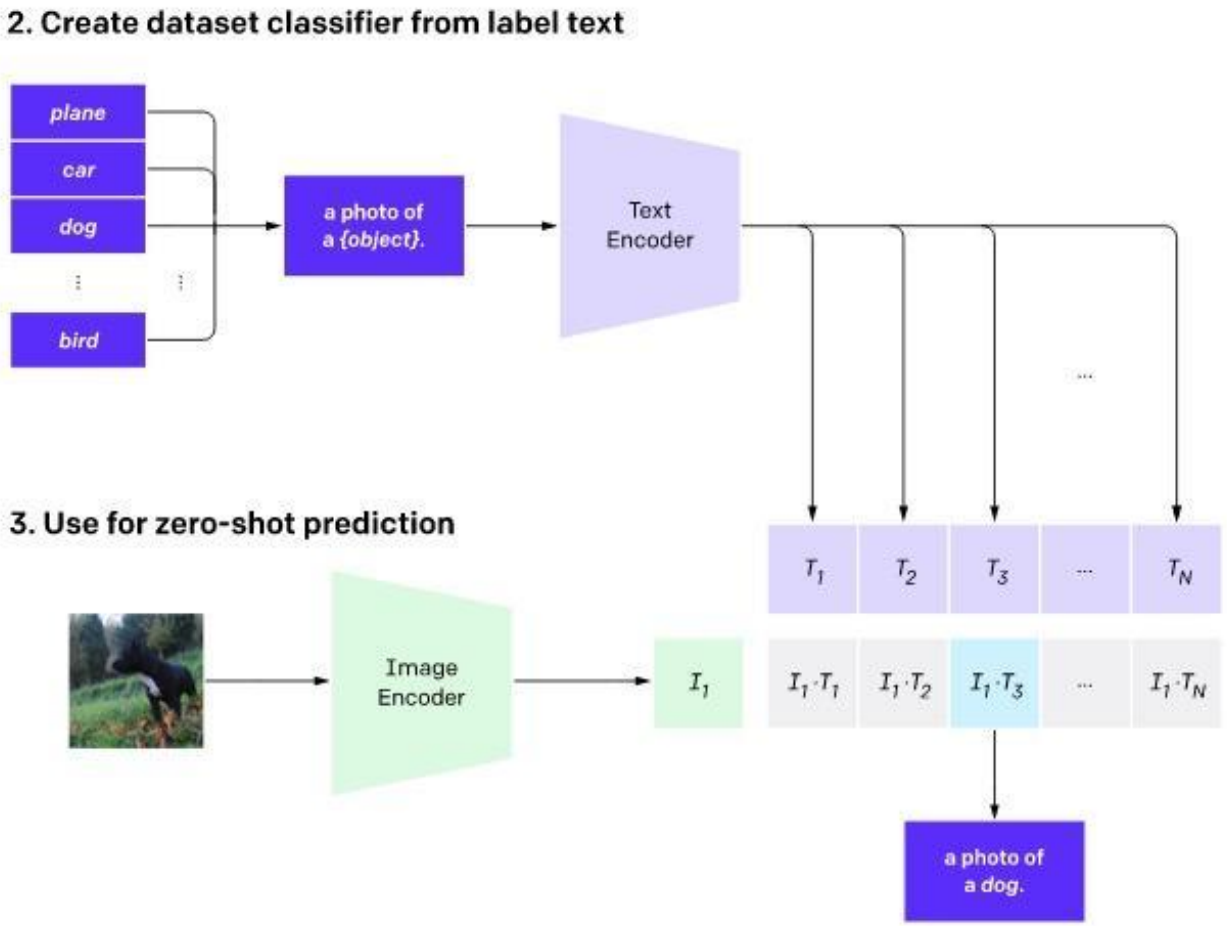
- Image encoding
  - ResNet-D — a strong CNN classifier — at several sizes
  - ViT — Vision Transformers, which treat small image patches as tokens

- Text encoding
  - Averaged CBOW embeddings (a simple baseline)
  - Transformer models

Final CLIP: ViT image encoder + transformer text encoder

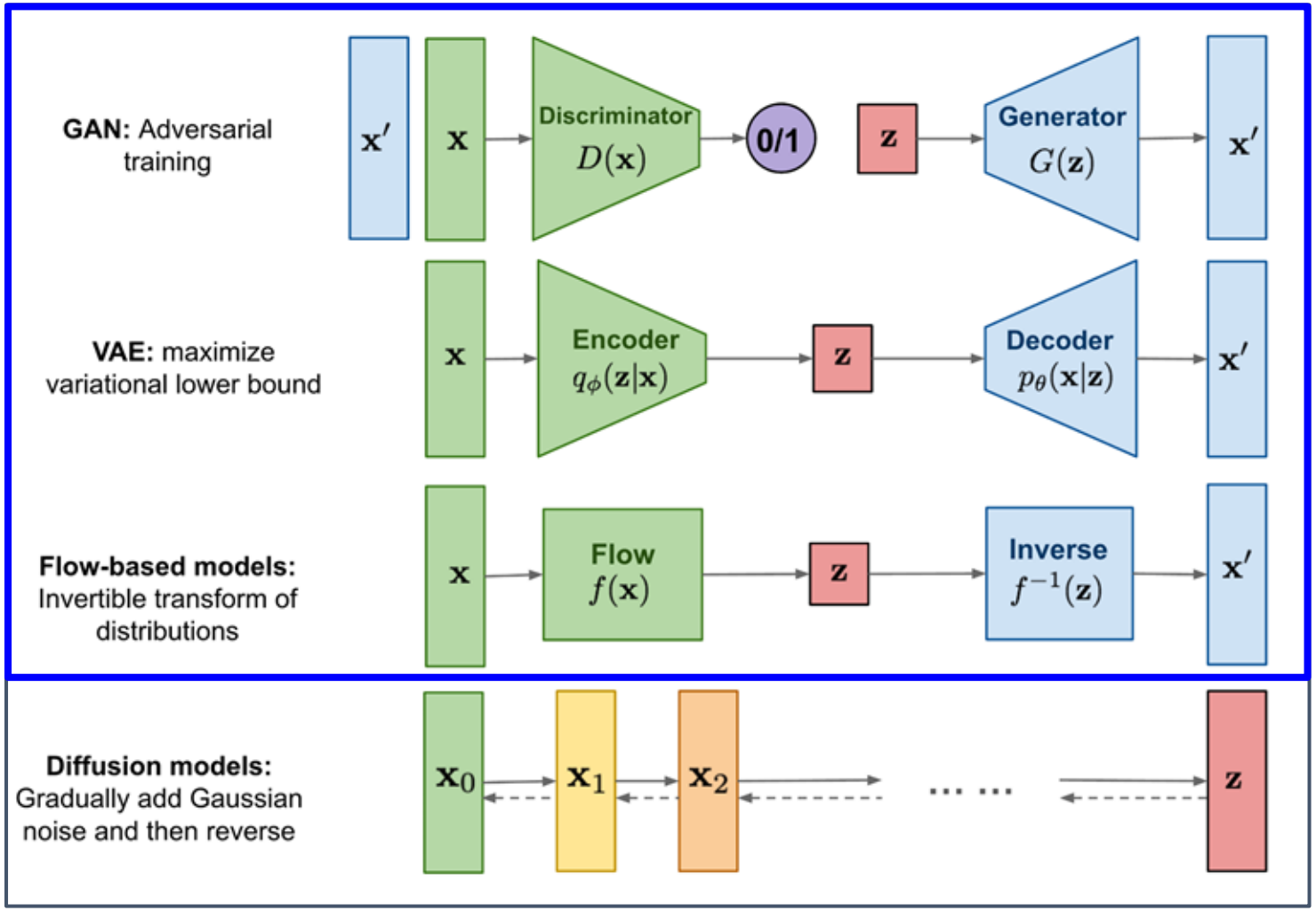
CLIP: Zero Shot Image Classification

- CLIP gives you a classifier for free: embed an image and a set of candidate captions, then SoftMax over the similarities. No extra training needed.
- Example: match the photo to “plane / car / dog / bird” — best fit is “dog,” no labels needed.



Learning Transferable Visual Models From Natural Language Supervision (https://arxiv.org/pdf/2103.00020.pdf)

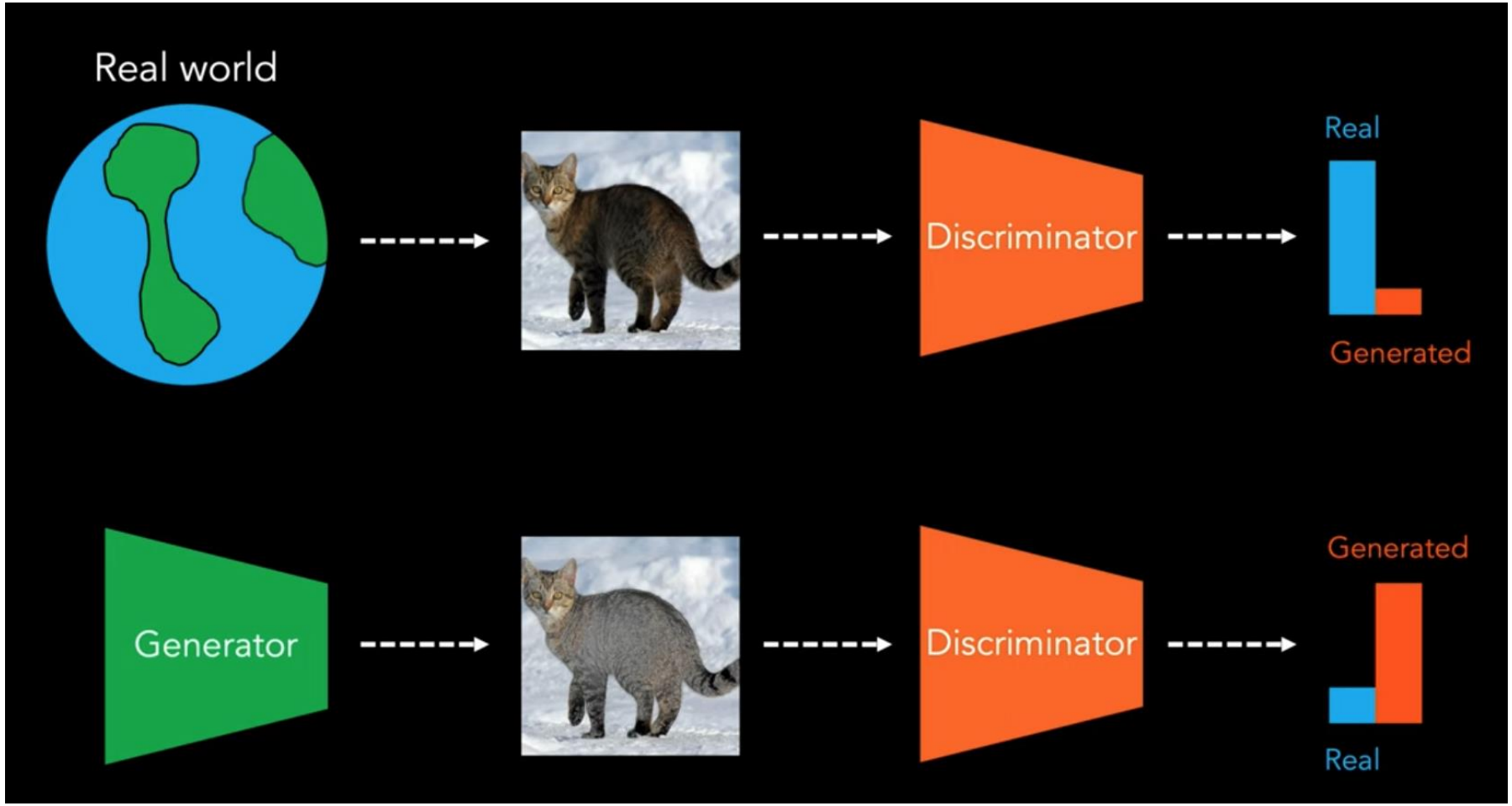
Four Popular Generative Models (2021)



- **Roadmap:** four families of generative models, all mapping noise → data. The blue box marks where we are.
- Refer to Appendix for approach details

*Image from https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

Generative Adversarial Networks



The analogy: a counterfeiter (generator) vs. a detective (discriminator). Both improve until the fakes look real.

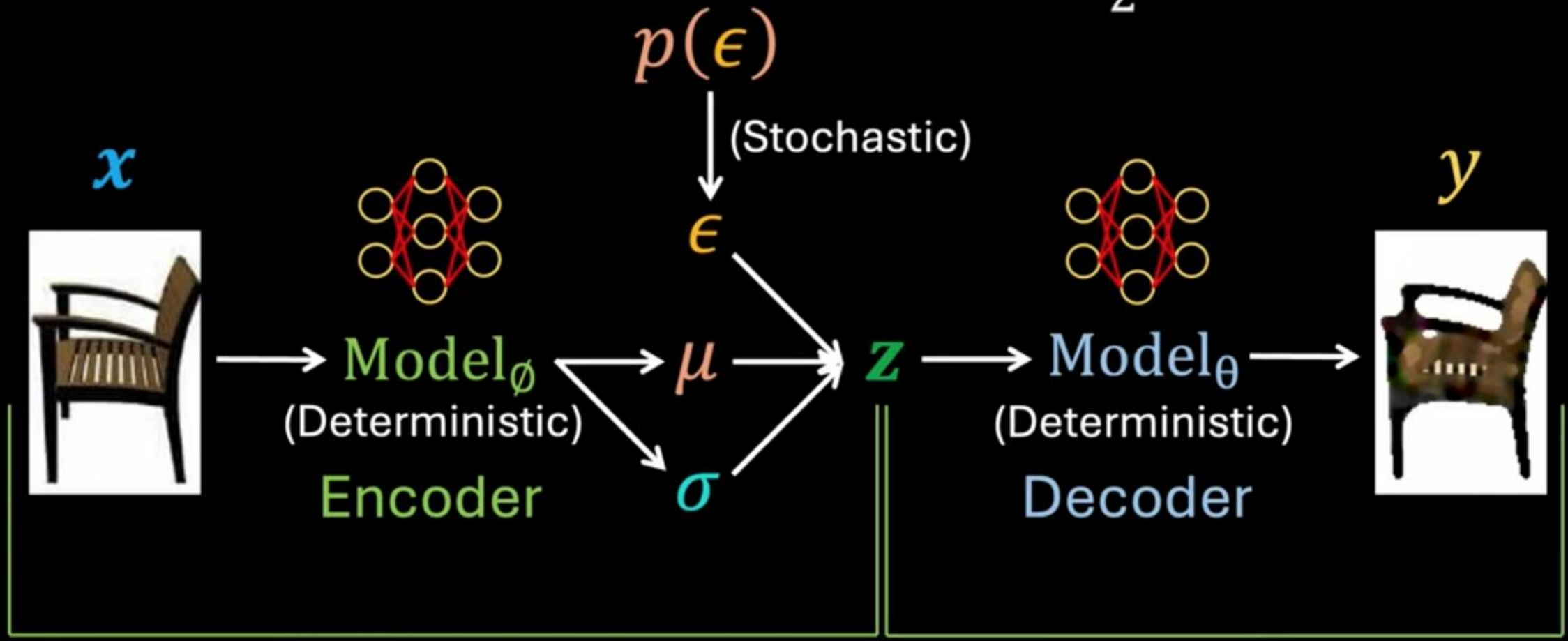
Understanding GANs (Generative Adversarial Networks)

Goodfellow et al., Generative Adversarial Networks (arXiv:1406.2661)

Variational autoencoder

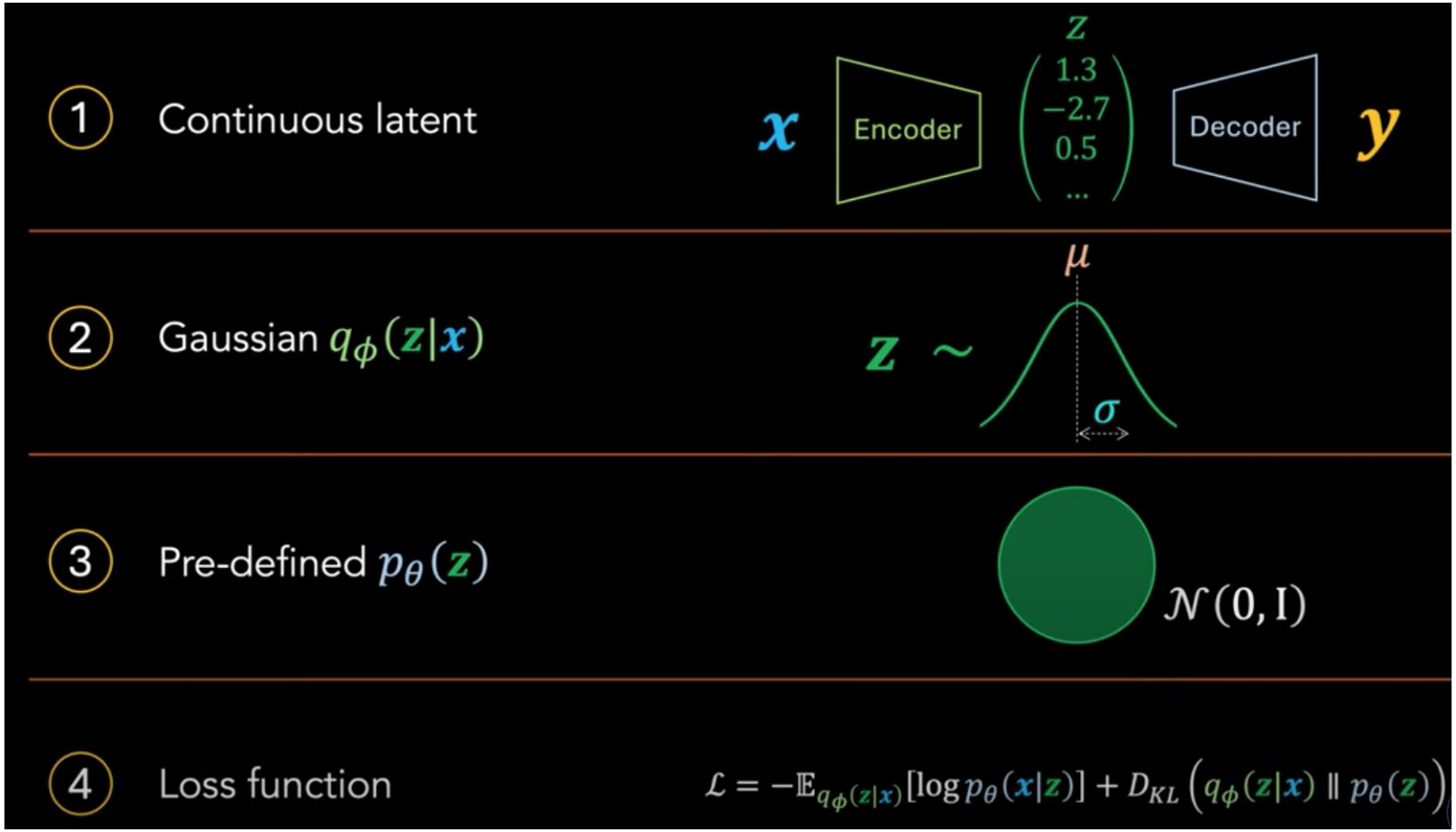
$$\mathcal{L}_{\text{recons}} = (x - y)^2$$

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2}[\log \sigma^2 - \mu^2 - \sigma^2 + 1]$$



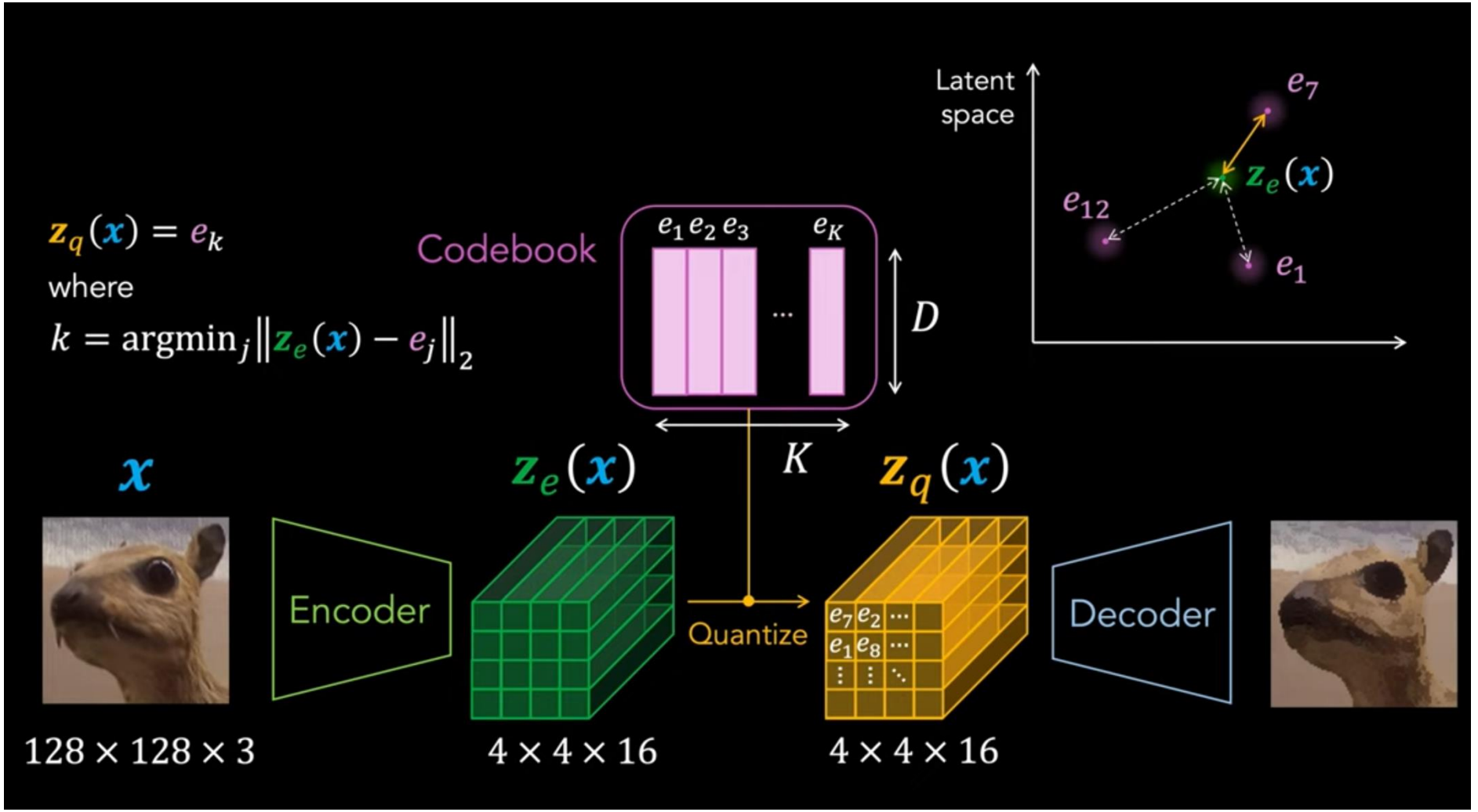
Kingma & Welling (arXiv:1312.6114)

Continuous VAE



Takeaway: a continuous Gaussian latent is smooth but can blur outputs — next, VQ-VAE makes the latent discrete.

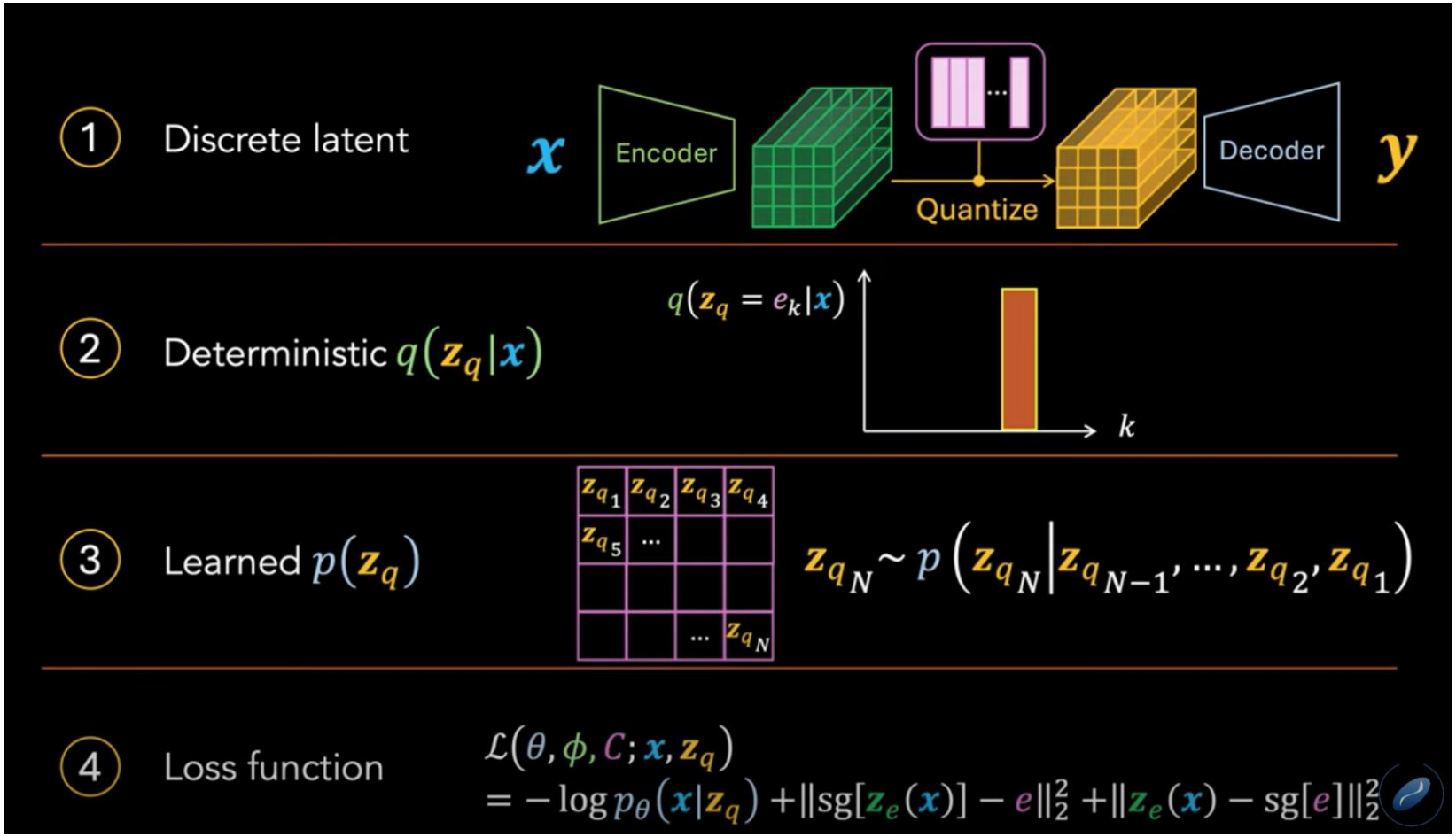
Vector-Quantized VAEs (VQ – VAE)

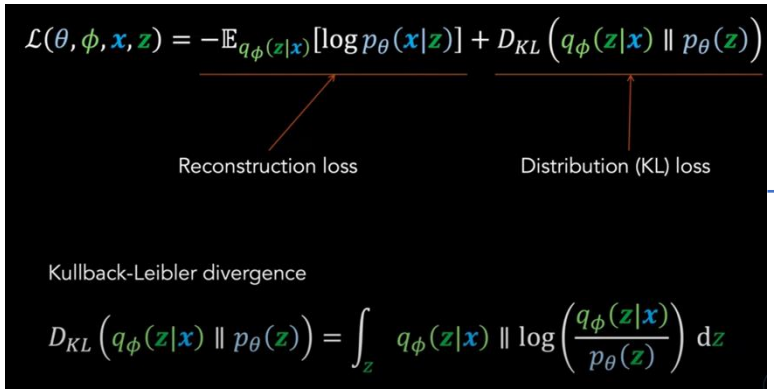


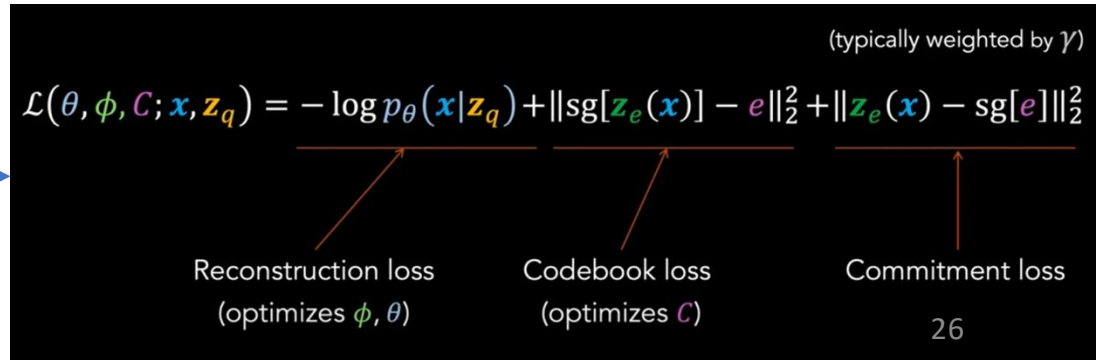
Vector-Quantized VAEs — van den Oord et al., Neural Discrete Representation Learning (arXiv:1711.00937)

VQ-VAE, *Cont.*

- **Key idea:** the codebook turns each image patch into one of a fixed set of “visual words” — so a transformer can model images the way it models text.







van den Oord et al. (arXiv:1711.00937)

Flow based Models

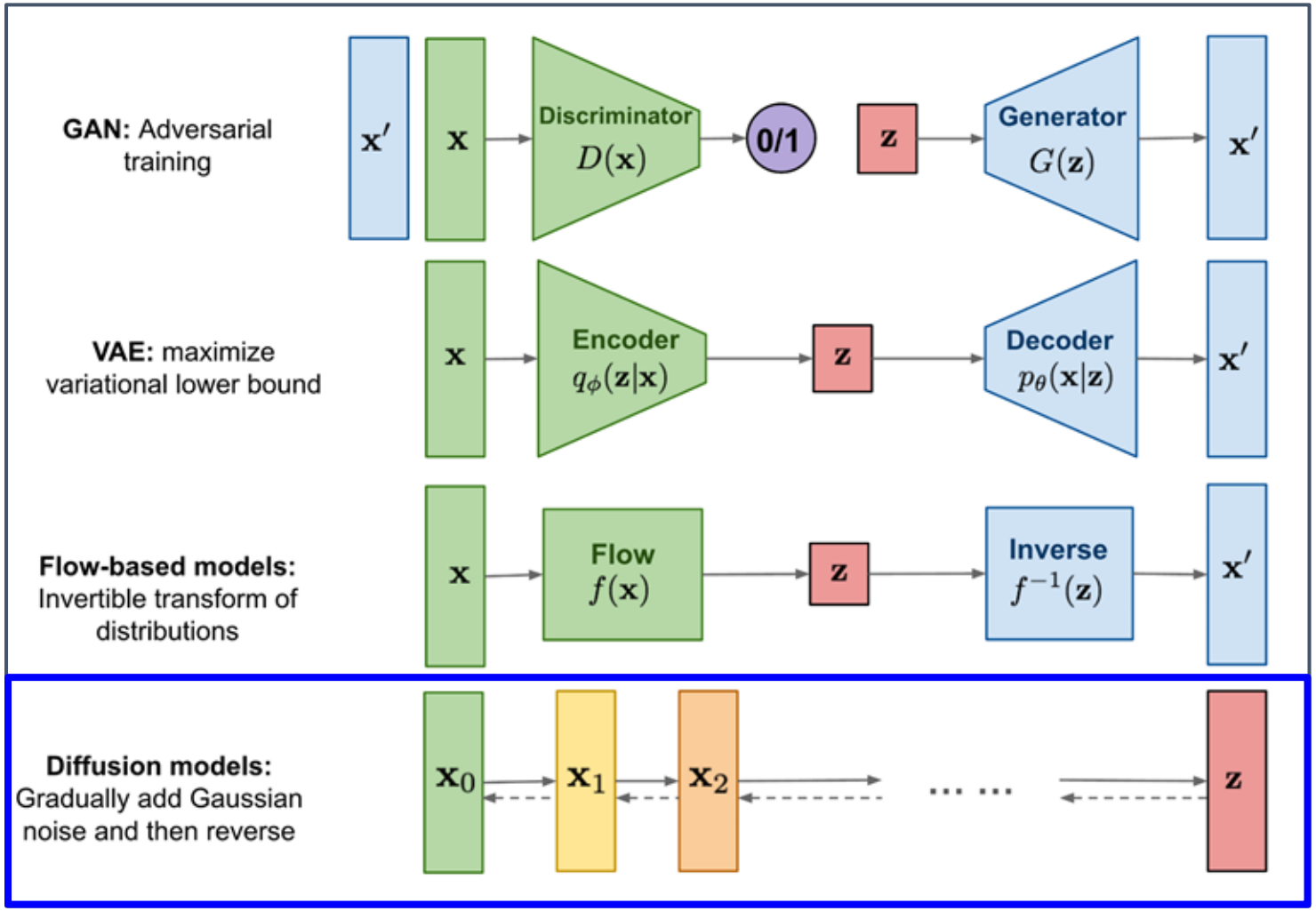
Flow models learn exact, invertible maps between noise and data.

- **Flow Matching —** learn a deterministic velocity field that sends noise straight to data — no diffusion-style step-by-step reversing.
  - *e.g., follow the straight "current" from noise to image.*
- **Masked Autoregressive Flow (MAF) —** transform values one at a time; each depends only on the earlier ones.
  - *e.g., generate each value from the ones before it — like writing word by word.*
- **Inverse Autoregressive Flow (IAF) —** the exact inverse of MAF — swap the conditioning so the flow runs the other way.
  - *e.g., the mirror of MAF: reversed conditioning makes sampling fast.*

MAF is fast to score; IAF is fast to sample — mirror images.

Lipman, Yaron, et al. "Flow matching guide and code." arXiv preprint arXiv:2412.06264 (2024).

Generative Paradigm: Diffusion Models

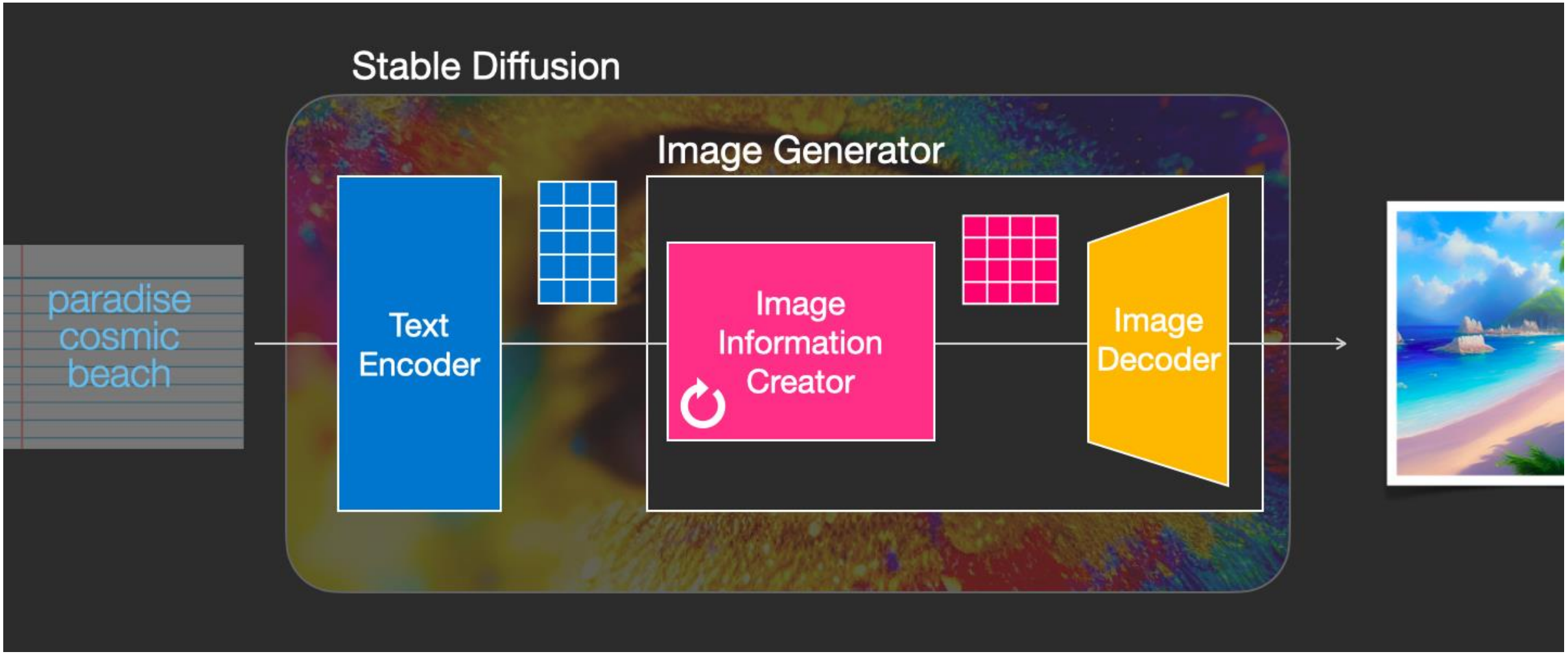


- **Our roadmap:** four families of generative models, all mapping noise → data. The blue box marks where we are.
- Diffusion models: gradually add noise to data, then learn to reverse the process, step by step.

*Image from https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

Ho et al., Denoising Diffusion Probabilistic Models (https://arxiv.org/abs/2006.11239)

Stable Diffusion



Rombach et al., High-Resolution Image Synthesis with Latent Diffusion Models (arXiv:2112.10752)

Figures: Jay Alammar, "The Illustrated Stable Diffusion"

Original image

Image Encoder

Generate training examples with different amounts of noise added to their compressed/latent version

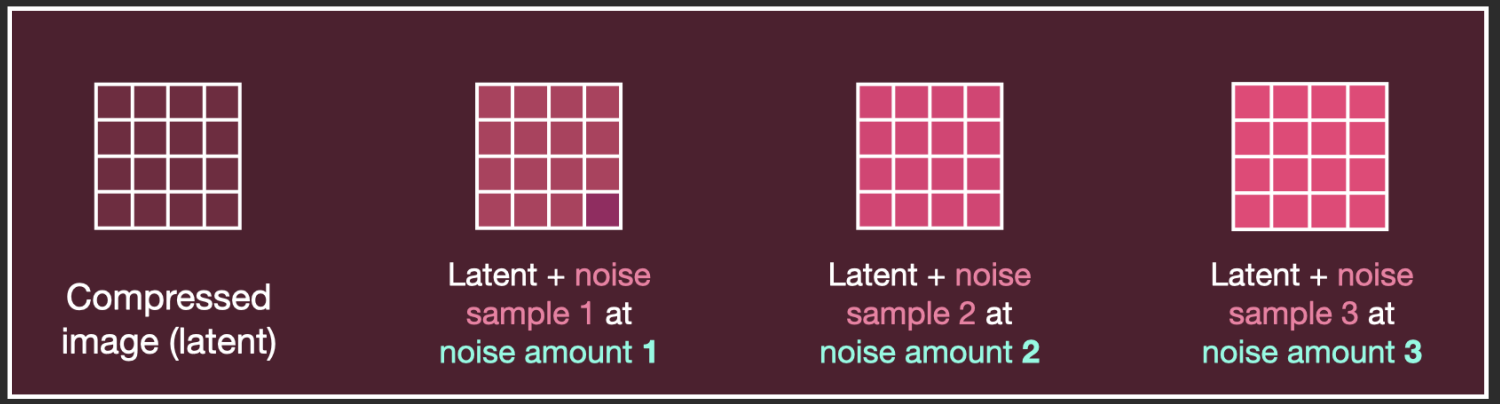


Image Generation by Reverse Diffusion (Denoising)

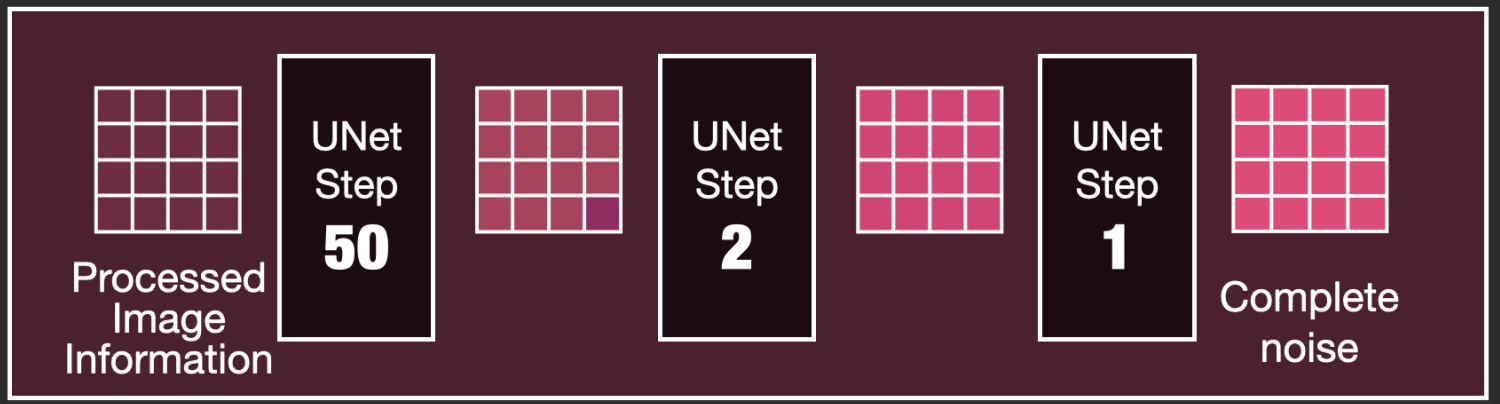


Image Information Creator

Image Decoder

Generated image

Figure: Jay Alammar, "The Illustrated Stable Diffusion" (jalammar.github.io)

Image Generation by Reverse Diffusion (Denoising)

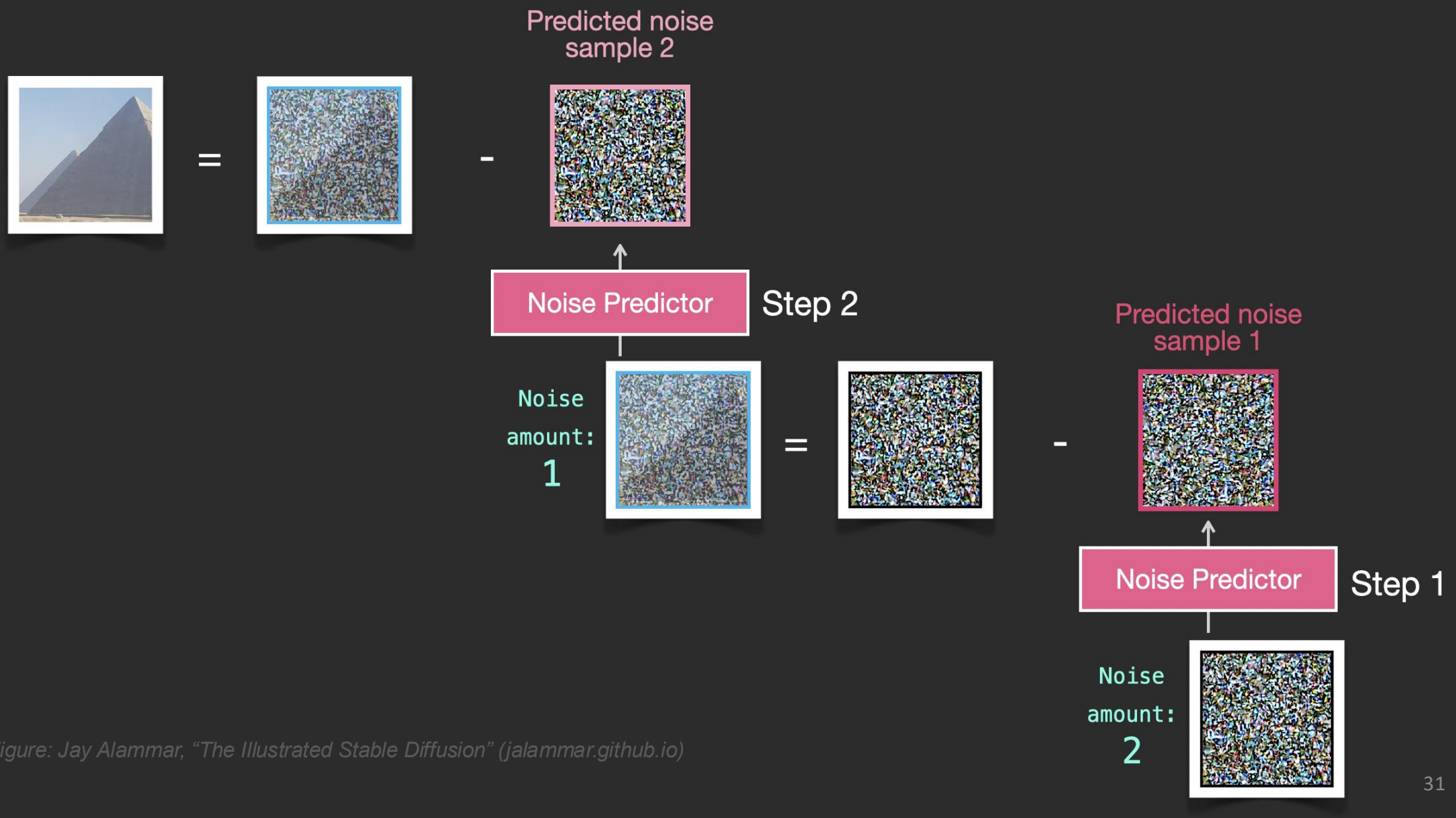
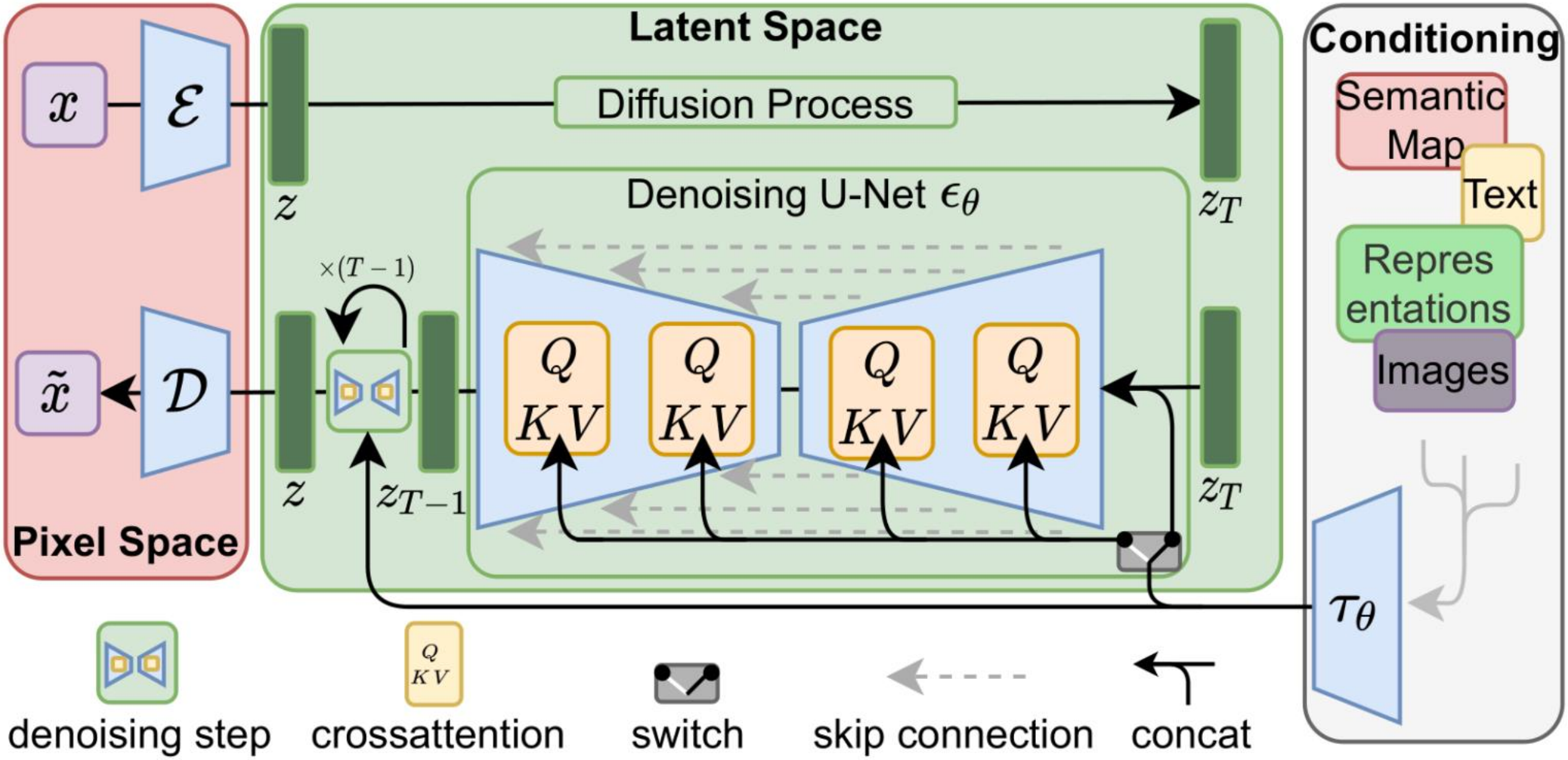


Figure: Jay Alammar, "The Illustrated Stable Diffusion" (jalammar.github.io)



Key idea: *diffusion runs in a compressed latent space — far cheaper than pixel space.*

(Rombach et al., arXiv:2112.10752)

Inside the U-Net: Adding Text via Attention

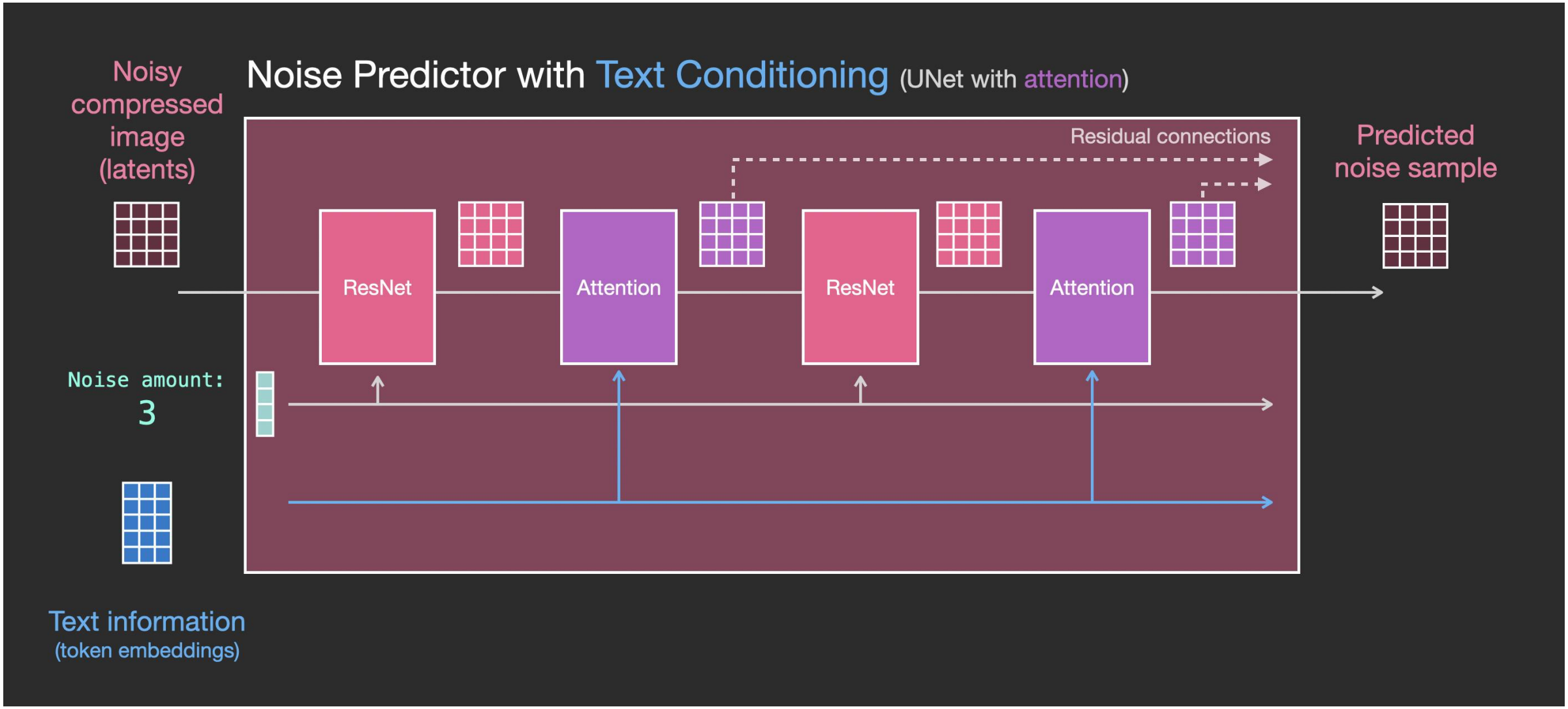
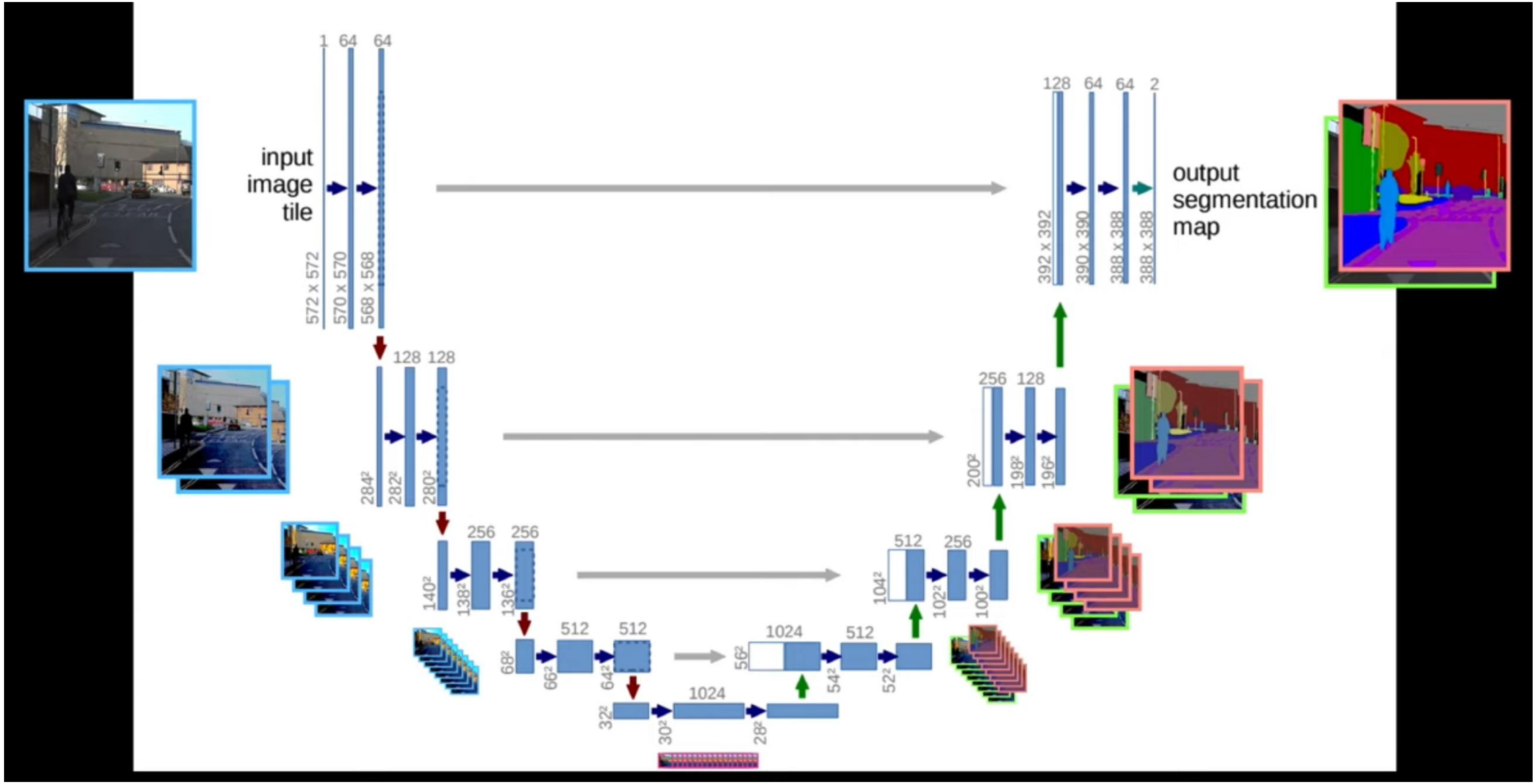


Figure: Jay Alammar, "The Illustrated Stable Diffusion"

U-Net: The Noise Predictor's Backbone



U-Net origin: Ronneberger et al. (arXiv:1505.04597)

Imagen (Google)

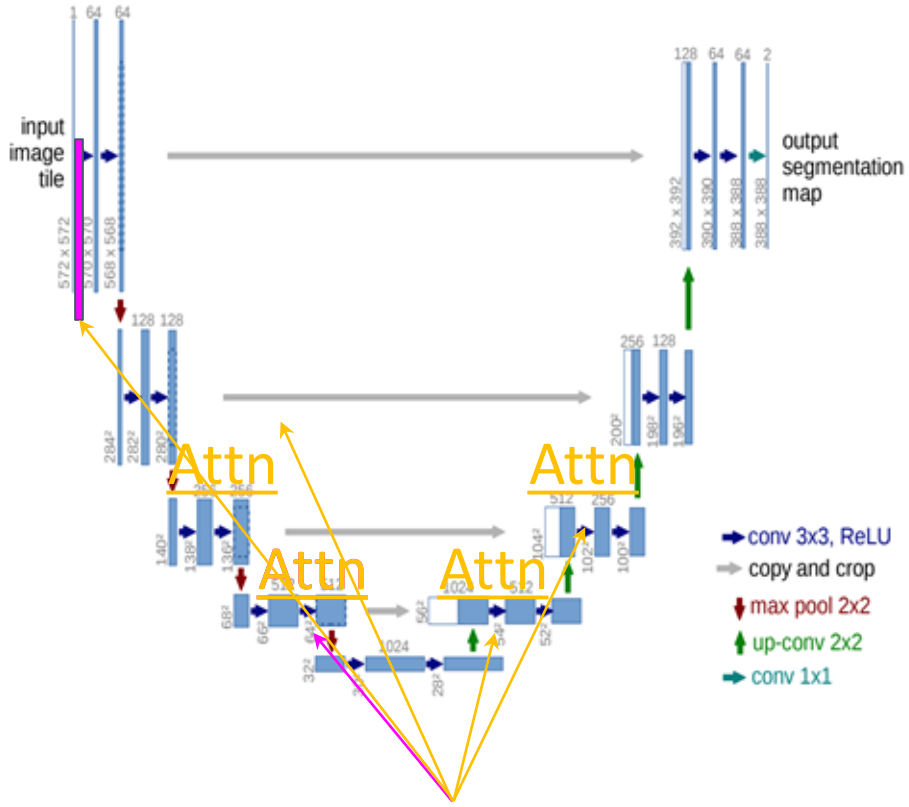


Imagen conditions its U-Nets on frozen T5 text embeddings — no CLIP needed. A plain language model was enough.

Imagen: Saharia et al., Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding (arXiv:2205.11487)

Part 3:

Video Generation

Evolution of Video Generative Foundations

How the goal of video generation shifted over five years — from motion, to quality, to physics.

2020–2021

GAN era

Goal: make things move. Keep objects steady frame-to-frame and cut the flicker.

2023

Diffusion era

Goal: text-to-video with sharper quality — powered by large text–image pretraining.

2024–2025

World models

Goal: go beyond visuals — simulate physics and the real rules of the world.

The big shift: from making pixels move → to modeling how the world actually works.

Foundations of Generative Paradigms

Three ways models turn noise into video — each trading off speed against quality.

Diffusion (DDPM)

Removes noise step by step — from pure static to a clean frame.

Trade-off: top quality, but slow — it needs many denoising steps.

Flow Matching

Draws a straight path from noise to data using vector fields.

Trade-off: fewer steps, so it's faster than diffusion.

Autoregressive (AR)

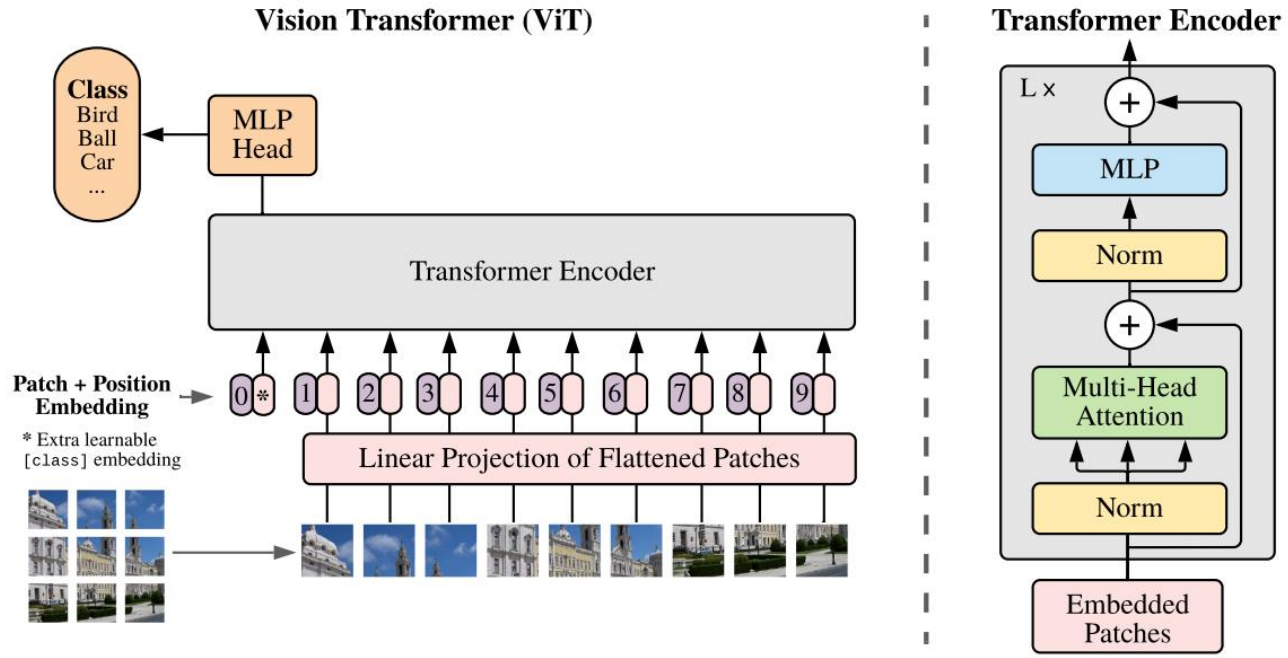
Predicts the next frame from all previous ones, like an LLM for video.

Trade-off: natural for causal video, but memory grows with length (KV-cache).

In practice all three run inside a Latent Diffusion (LDM) framework — on a compressed latent, not raw pixels. Handling time is the hard part.

M. He, H. Guo, J. Lin, and Y. Yu, "Video Generation Models as World Models: Efficient Paradigms, Architectures and Algorithms," arXiv:2603.28489, 2026.

ViT: An Image Is Worth 16×16 Words



Key idea: image patches = tokens. With enough data, ViT (Vision Transformer) beats CNNs — and became CLIP's image encoder.

Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Dosovitskiy et al., An Image Is Worth 16×16 Words (arXiv:2010.11929)

Visual Autoregressive Modeling (VAR)

Key idea: *predict the next resolution, not the next patch — coarse to fine, like sketching before refining. Tian et al. (arXiv:2404.02905), NeurIPS 2024 Best Paper*

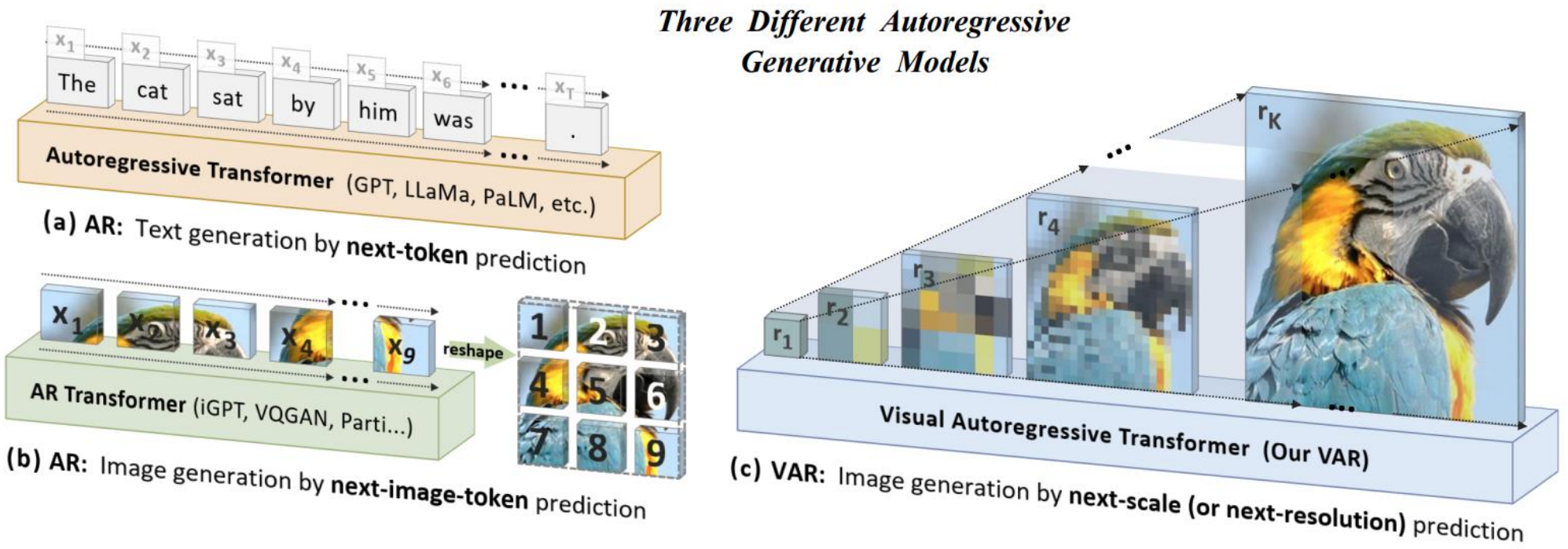


Figure 2: **Standard autoregressive modeling (AR) *vs.* our proposed visual autoregressive modeling (VAR).** (a) AR applied to language: sequential text token generation from left to right, word by word; (b) AR applied to images: sequential visual token generation in a raster-scan order, from left to right, top to bottom; (c) VAR for images: multi-scale token maps are autoregressively generated from coarse to fine scales (lower to higher resolutions), with parallel token generation within each scale. VAR requires a multi-scale VQVAE to work.

VideoPoet: Zero-Shot Video Generation

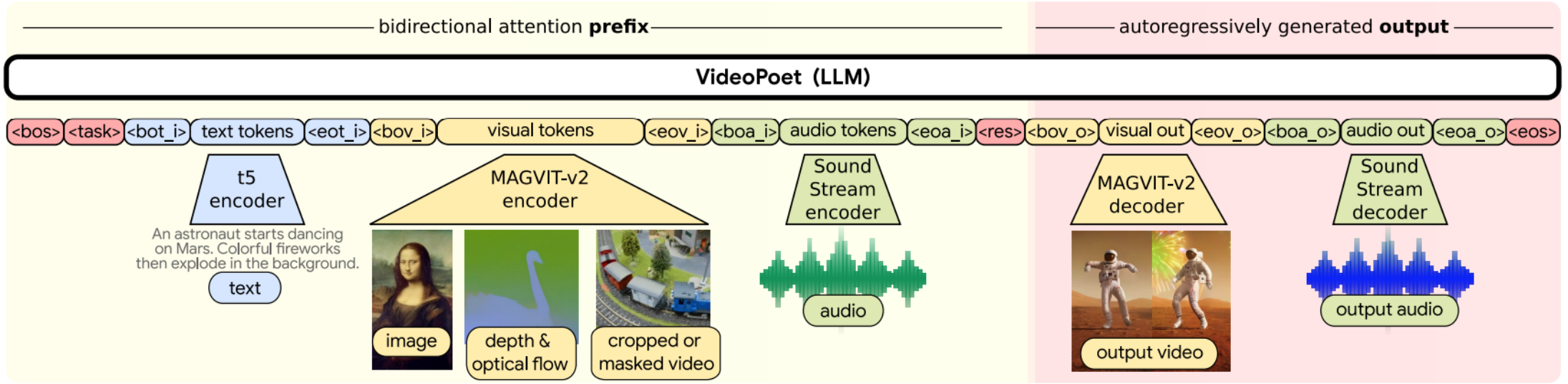


Figure 2: **Sequence layout for VideoPoet**. We encode all modalities into the discrete token space, so that we can directly use large language model architectures for video generation. We denote special tokens in <> (see Table 4 for definitions). The modality agnostic tokens are in darker red; the text related components are in blue; the vision related components are in yellow; the audio related components are in green. The left portion of the layout on light yellow represents the bidirectional prefix inputs. The right portion on darker red represents the autoregressively generated outputs with causal attention.

Key idea: *turn every modality — text, image, video, audio — into tokens, and one LLM generates them all.*

Kondratyuk et al. (arXiv:2312.14125)

Sora: How Large Vision Models Generate Video

3.1 Overview of Sora

In the core essence, Sora is a diffusion transformer [4] with flexible sampling dimensions as shown in Figure 4. It has three parts: (1) A time-space compressor first maps the original video into latent space. (2) A ViT then processes the tokenized latent representation and outputs the denoised latent representation. (3) A CLIP-like [26] conditioning mechanism receives LLM-augmented user instructions and potentially

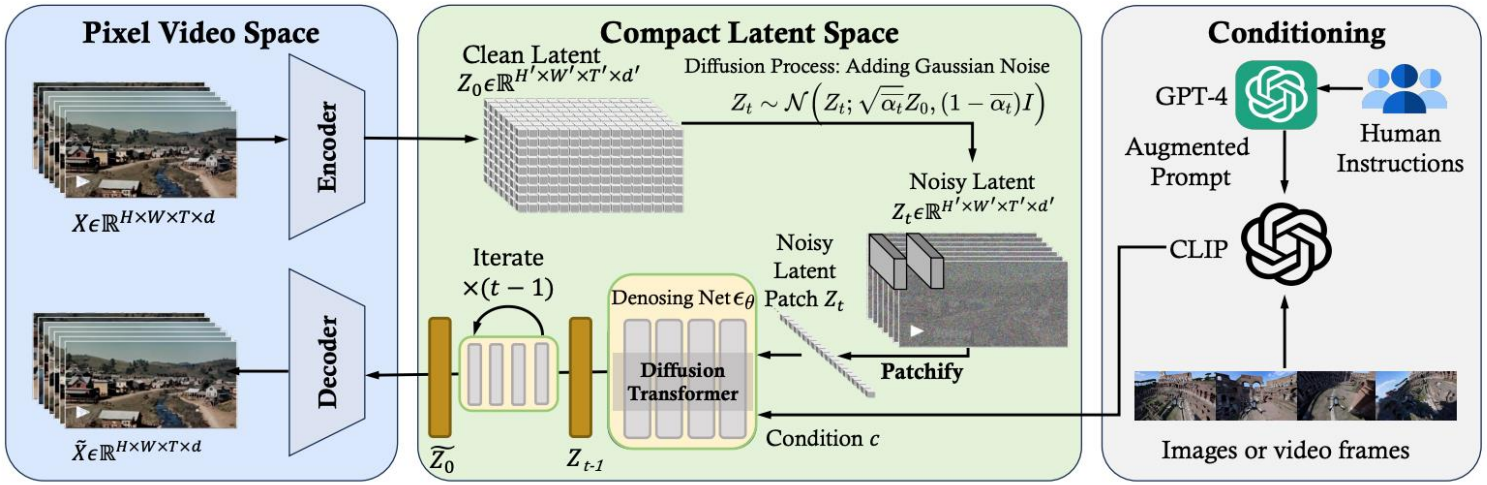


Figure 4: **Reverse Engineering**: Overview of Sora framework

Key idea: video becomes spacetime patches — the ViT trick, extended through time.

Liu et al., Sora: A Review (arXiv:2402.17177) • Sora: OpenAI, 2024

The Temporal Gap and Core Challenges

Why video is harder than images — the "Temporal Gap."

- **Consistency & causality —** objects can't flicker, morph, or vanish, and must obey physics like gravity and collisions.
  - *e.g., a character's red shirt stays red for all 300 frames.*
- **Compute explosion —** adding a time axis blows up data roughly cubically; needs efficient designs like Causal VAEs.
  - *e.g., one 5-second clip = hundreds of frames modeled together.*
- **Data & caption scarcity —** high-quality captioned video is rare, so models use LMMs to auto-write captions.
  - *e.g., an image needs alt-text; a video needs motion + camera + events.*

Bottom line: stay consistent over time, at cubic compute cost, with far less labeled data.

Long-Horizon Interactions & Memory Mechanisms

The problem: in long videos, small errors pile up — the scene "drifts" out of consistency.

- **Memory — remember the past:** store history so earlier objects stay put.
  - **Implicit memory:** keep past video tokens in a bank; recall them by place or time (RELIC).
  - **Explicit 3D storage:** pin the world as 3D geometry (Gaussian Splatting, Surfels) so an object looks the same from any angle (Vmem).
- **Forcing — train for stability:** teach the model to survive long rollouts.
  - **Diffusion Forcing:** denoise from a noisy history, so it learns to fix its own mistakes.
  - **Context Forcing:** a long-context "teacher" coaches an efficient "student" to stay consistent.

e.g., drive past a building, turn around — memory keeps it there, unchanged.

Efficient Modeling Paradigms

Two ways to make video generation efficient — fewer steps, and endless streaming.

- **Fewer steps: distillation —** cut the many denoising steps down to one or a few.
  - **Consistency Distillation:** a “student” jumps straight to the final frame in one step.
  - **Adversarial Distillation:** adds a critic to keep those few-step frames sharp.
  - *e.g., 50 denoising steps → just 1–4.*
- **Endless video: causal streaming —** build each new moment from the past only, for any length.
  - **Diffusion / Rolling Forcing:** slide a window forward — drop the oldest frame, add a new one.
  - **Bounded state:** clips stay stable for minutes, with no freezing or motion decay.
  - *e.g., a treadmill — always add the next frame, forget the oldest.*

Interactive Video World Modeling

A world model learns how an environment changes — and predicts what happens next.

- **Traditionally — passive prediction:** learn the dynamics, then forecast future frames from past observations.
- **Now — interactive world modeling:** AIGC (Artificial Intelligence Generated Content) + multimodal LLMs fold the user's actions into the state transition.
- **The shift:** from a passive prediction engine → to an active, controllable environment you can steer.

  e.g., not just watching a car drive, but pressing "turn left" and the world responds.

Evolution of Research Trends

Three ways world-model research is maturing:

- **Specialist → generalist:** narrow single-task models give way to broad ones built on pretrained video backbones (UniSim, GAIA-1) that cross domains.
- **Static → dynamic worlds:** explorable-but-frozen 3D scenes become self-evolving worlds with "out-of-sight" changes.
- **Richer interaction — beyond text & visuals:**
  - **Audio:** sound shapes the scene (SonoWorld).
  - **Physical force:** gravity, friction, collisions (RealWonder).
  - **Structured data:** driving inputs like ego-velocity and road semantics.

e.g., leave a room, return later — time has passed and things have moved.

Technical Bottlenecks: User Action Controllability

"Action injection" = how a user's control signal (like a camera move) enters the generation.

- **Concatenation —** turn the action into tokens and paste them alongside the video tokens (iVideoGPT).
- **Scaling & shifting —** let the action tweak the network's normalization/attention to "steer" features (LingBot-World).
- **Camera-controlled rendering —** the action moves an explicit 3D model (point clouds, Gaussian Splats) before drawing the frame.
- **Matrix transformation —** use homography / panoramic math so camera moves stay geometrically correct.

  e.g., you press "pan right" — each method is a different route that keystroke takes to the pixels.

State-of-the-Art Foundation Models

Three training strategies trade off quality, cost, and openness:

- **World Foundation ($60–80M):** NVIDIA Cosmos, OpenAI Sora — exascale data + massive compute for top physical realism.
- **Balanced Research ($2–3M):** Seaweed-7B, CogVideoX — smart architectures and efficient attention.
- **Efficient Commercial ($200–400k):** open-source Open-Sora 2.0 — aggressive data curation + progressive low→high-res training.

  e.g., same goal, ~300× cost range — from ~$80M down to ~$200k.

Hu, Teng, et al. "Evolution of video generative foundations." arXiv preprint arXiv:2604.06339 (2026).

Downstream Applications and World Simulators

How video foundation models get adapted to real tasks:

- **Video Customization:** keep a person or object's identity from a reference image in new scenes.
- **Motion Control:** steer camera moves (pan, tilt, zoom) and object motion via sketches or coordinates.
- **Video Editing:** change style or add/remove objects via text, keeping the background consistent.
- **World Models:** predict "what happens next" for embodied AI and driving sims — an implicit physics engine.

From an entertainment tool → to a simulator of the physical world.

Appendix

GAN Objective Function

Value function

$$V(G,D) = \mathbb{E}_{x\sim p^*}[\log D(x)] + \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D(G(z))\right)\right]$$

"Minimax game"

$$\min_G \max_D V(G,D)$$

$$L_G = -L_D = V(G,D)$$

where

"strategy" = model parameters

In words: *D* tries to spot fakes, *G* tries to fool *D* — the same value function, played in opposite directions.

GAN Objective Function, *Continued*

$$V(G,D) = \mathbb{E}_{x\sim p^*}[\log D(x)] + \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D(G(z))\right)\right]$$

Find Nash equilibrium

1. Find optimal D give G (called D_G^*)

$$D_G^*(x) = \frac{p^*(x)}{p^*(x) + p_G(x)}$$

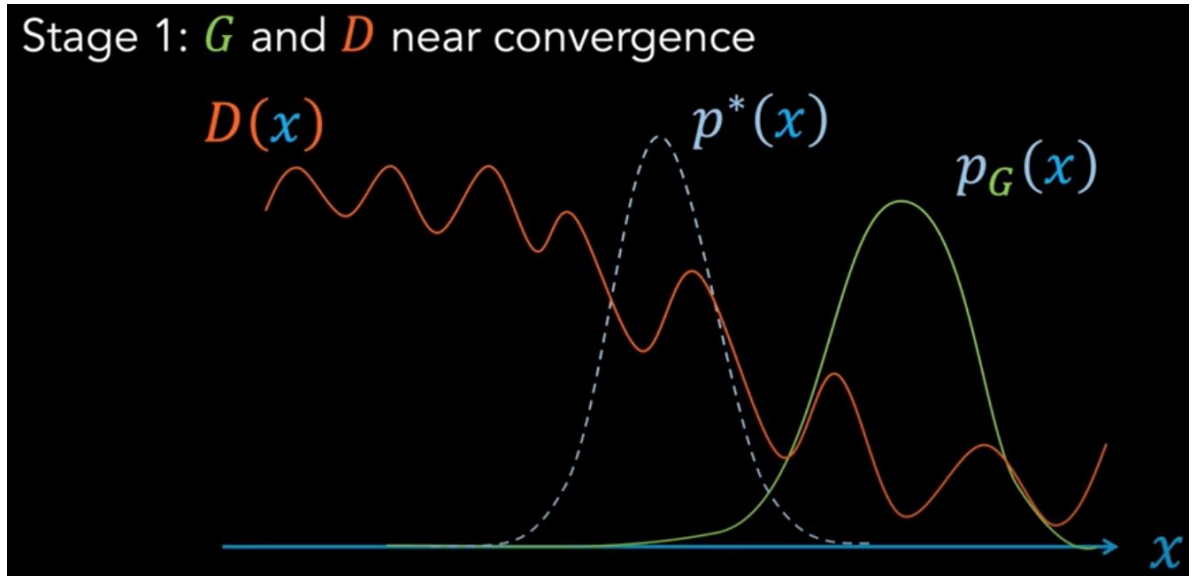
2. Find G assuming $D = D_G^*$

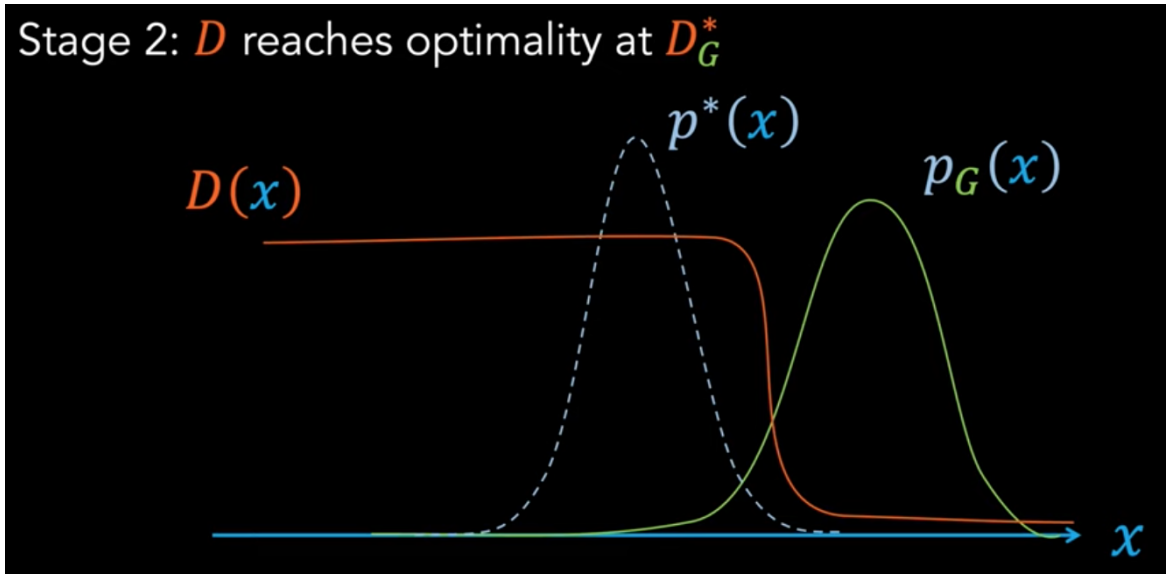
$$D_{JS}\left(p^*(x) \,\|\, p_G(x)\right) = 0$$

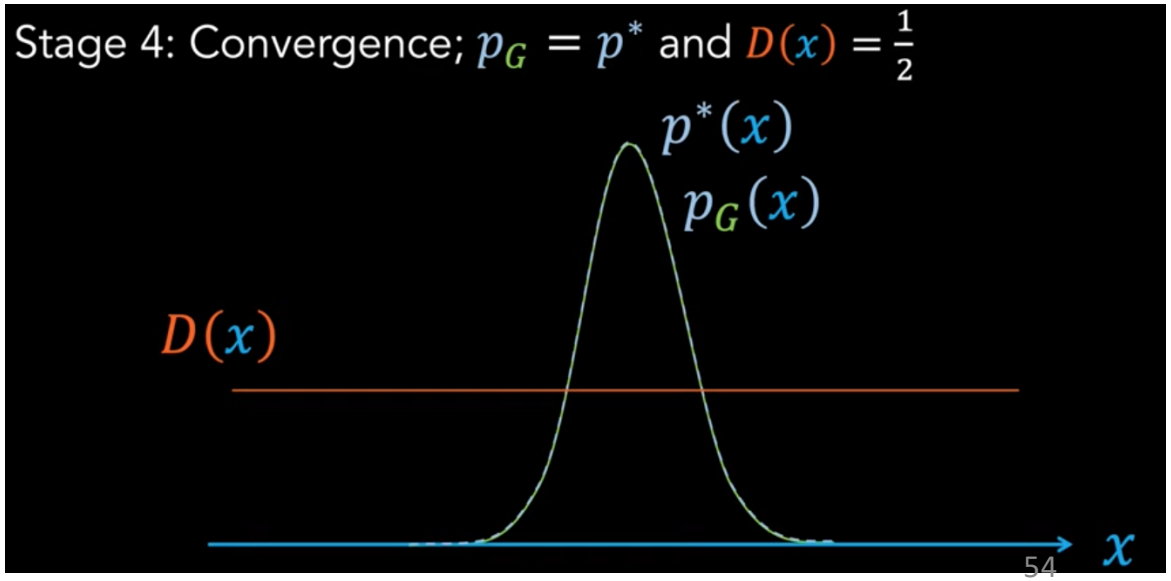
Prove $p_G \to p^*$

Takeaway: at the optimum the generator's distribution matches the real data: $p_G = p^*$.

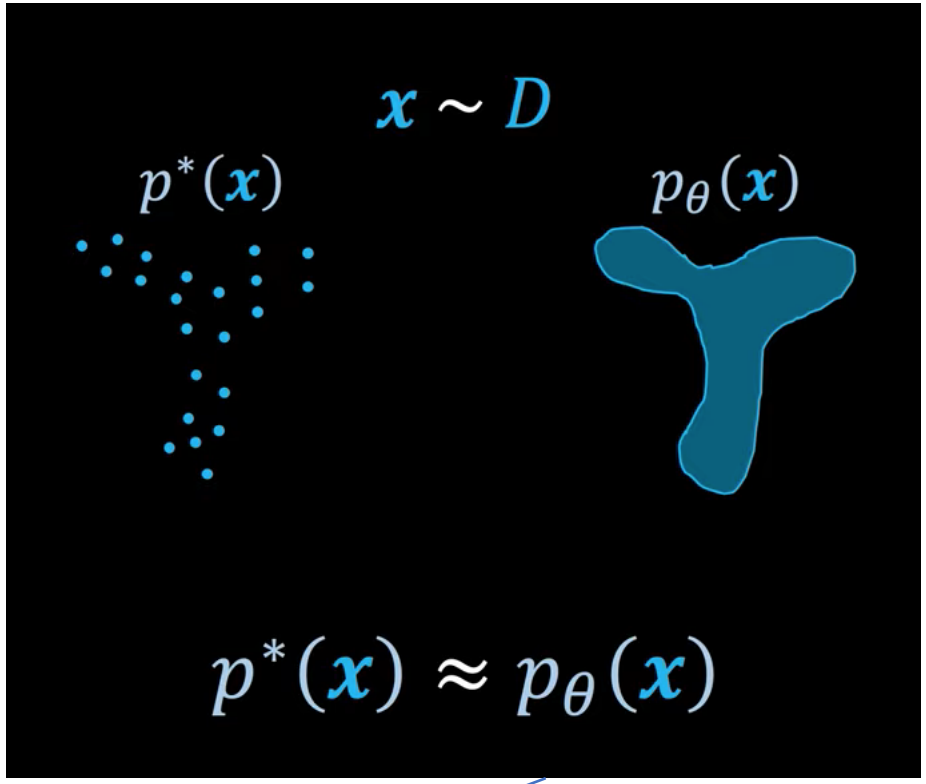
GAN Training

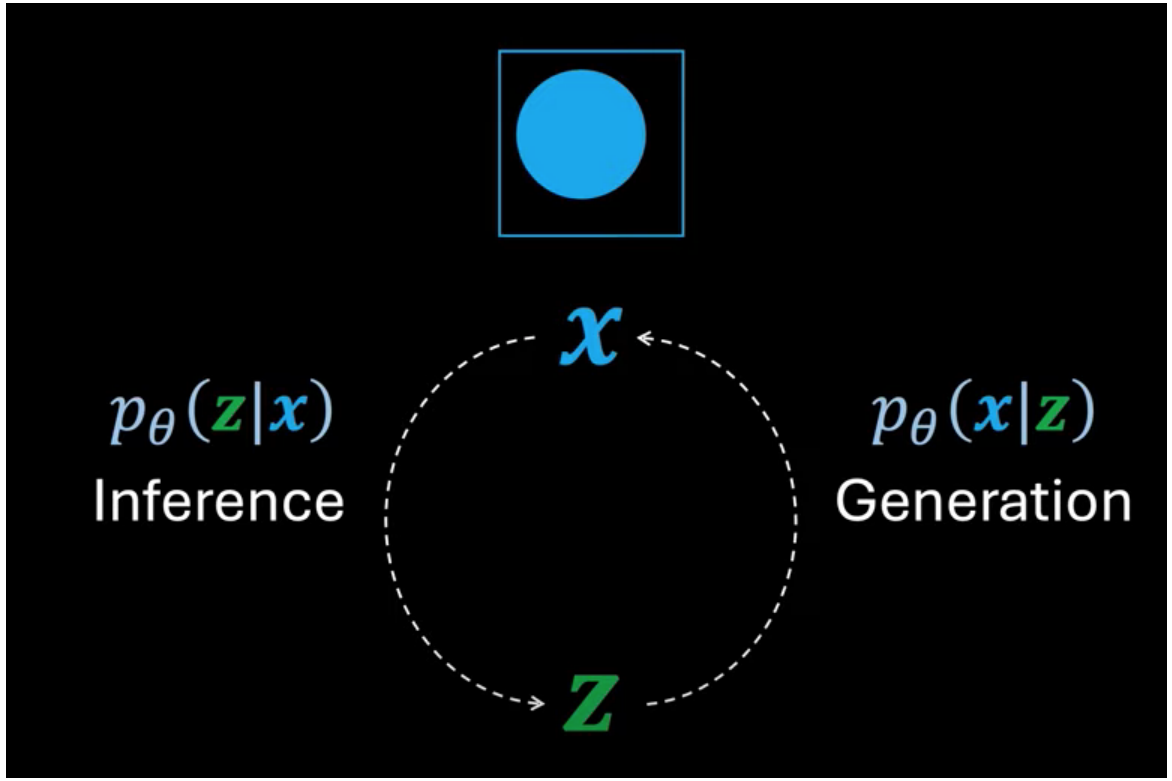






VAE (Variational Autoencoder)





Key idea: the encoder maps x to a distribution over latent z; the decoder maps z back to data. Sampling z yields brand-new images.

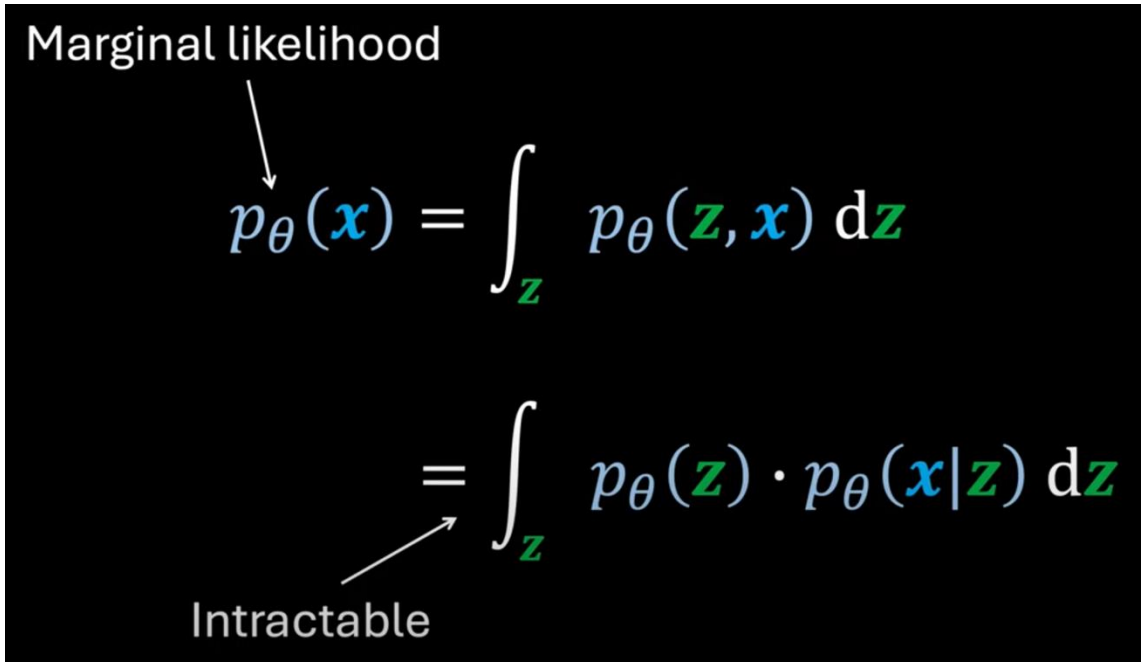
$P_\theta(x)$ is called the likelihood of data x given the model parameters θ.

Understanding Variational Autoencoders (VAEs)

Kingma & Welling, Auto-Encoding Variational Bayes (arXiv:1312.6114)

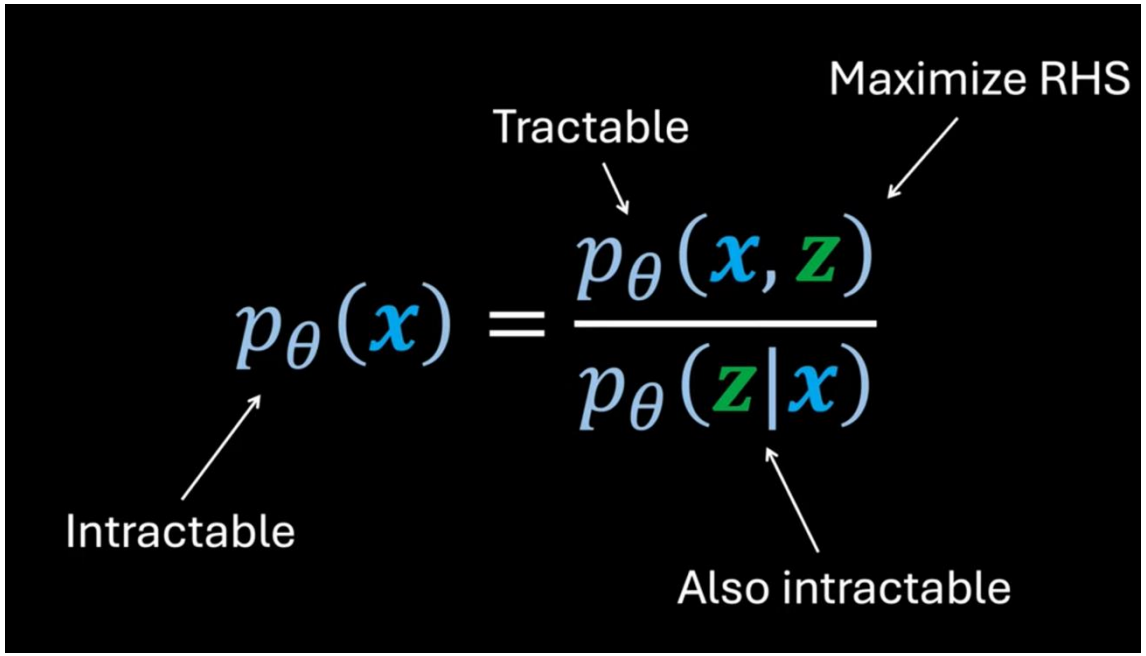
Why VAE Training Is Hard

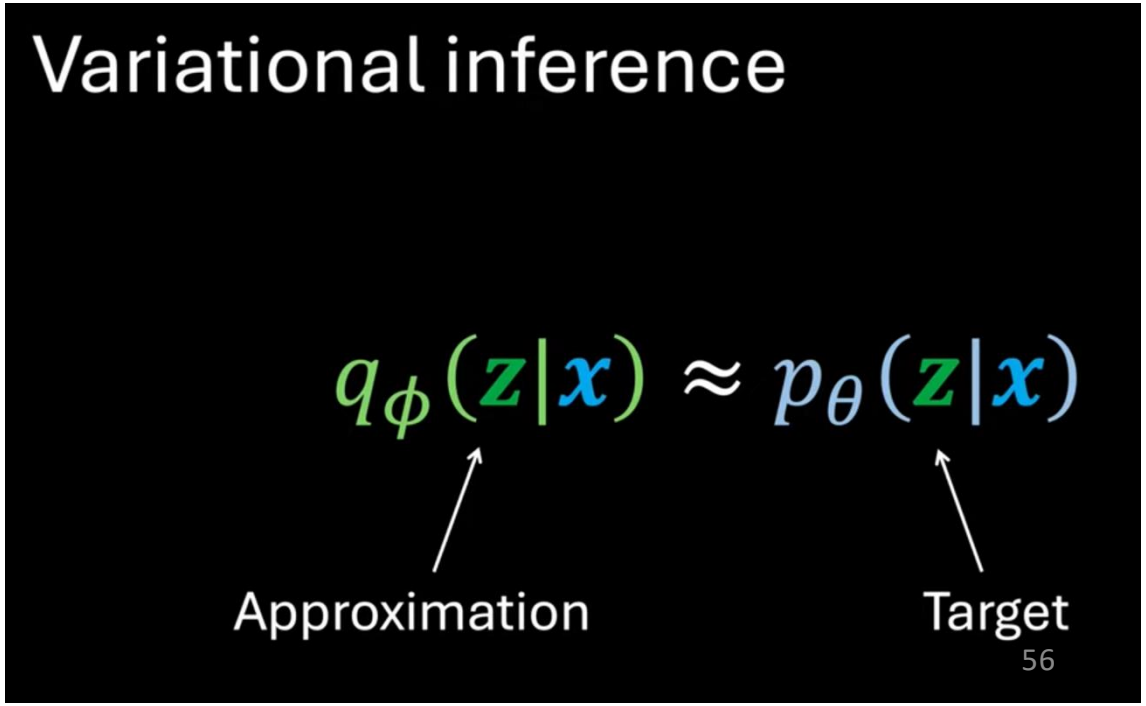
- The likelihood integral is intractable — so we approximate it with variational inference.



Key idea: $q_\varphi(z|x)$ is a learnable stand-in for the true (unknowable) posterior $p_\theta(z|x)$.

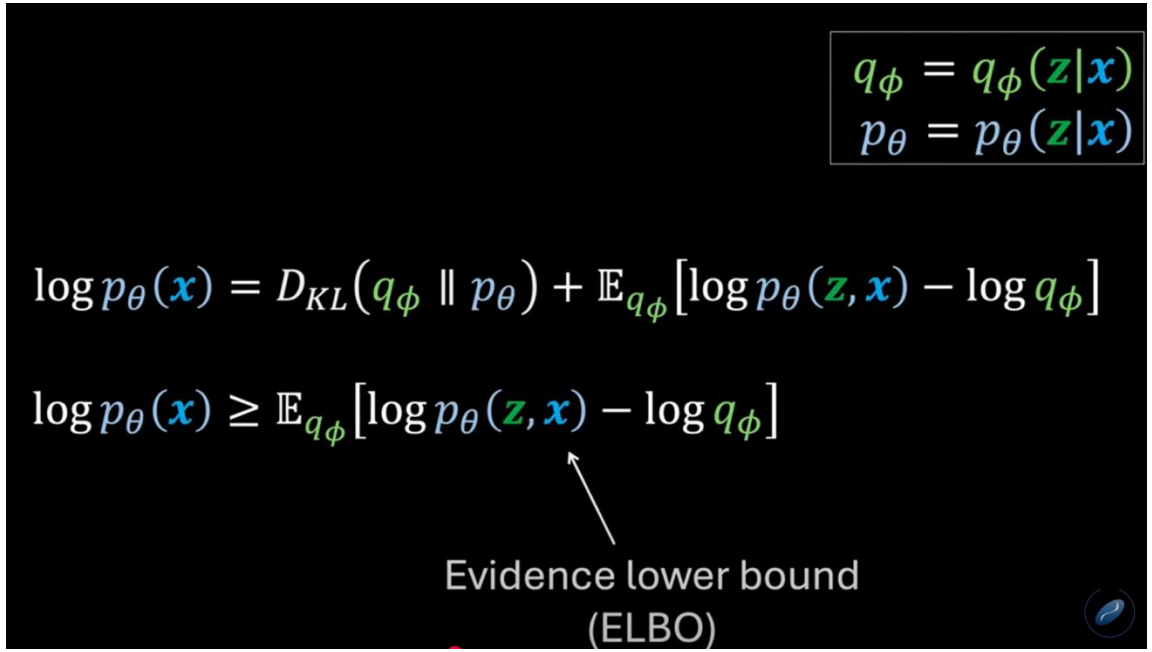
Kingma & Welling (arXiv:1312.6114)

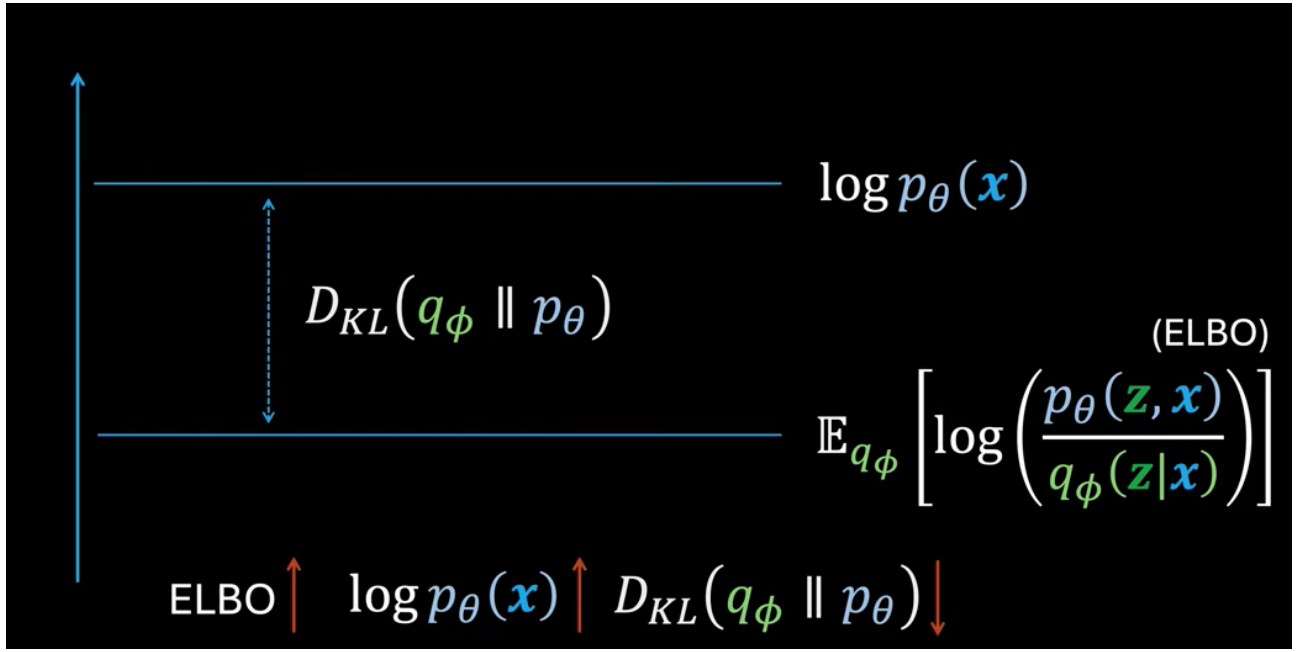




Evidence Lower Bound (ELBO)

We can’t maximize log p(x) directly, so we maximize a lower bound instead. The gap is exactly a KL divergence.



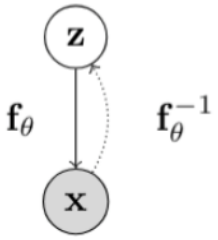


Kingma & Welling, Auto-Encoding Variational Bayes (arXiv:1312.6114)

Flow based Models

- A normalizing flow turns a simple distribution (e.g., a Gaussian) into a complex one by chaining many small, invertible transformations.

  - Consider a directed, latent-variable model over observed variables X and latent variables Z
  - In a **normalizing flow model**, the mapping between Z and X, given by $\mathbf{f}_\theta : \mathbb{R}^n \mapsto \mathbb{R}^n$, is deterministic and invertible such that $X = \mathbf{f}_\theta(Z)$ and $Z = \mathbf{f}_\theta^{-1}(X)$



  - Using change of variables, the marginal likelihood $p(\mathbf{x})$ is given by

$$p_X(\mathbf{x};\theta) = p_Z\left(\mathbf{f}_\theta^{-1}(\mathbf{x})\right)\left|\det\left(\frac{\partial \mathbf{f}_\theta^{-1}(\mathbf{x})}{\partial \mathbf{x}}\right)\right|$$

  - Note: $\mathbf{x}, \mathbf{z}$ need to be continuous and have the same dimension.

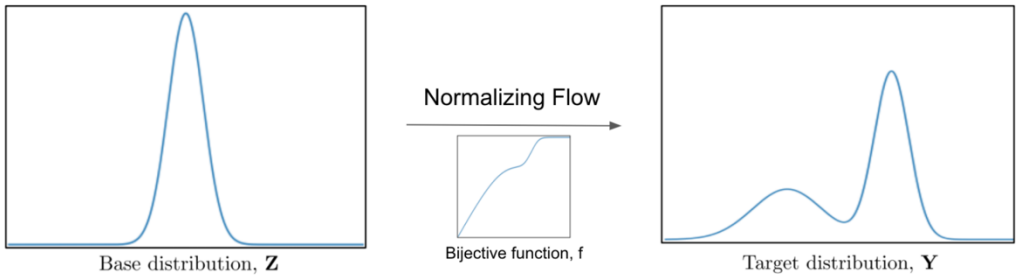


Fig 1: The transformation of a base distribution into a target distribution using a bijective function f.

Rezende & Mohamed, Variational Inference with Normalizing Flows (arXiv:1505.05770)

Masked Autoregressive Flow (MAF)

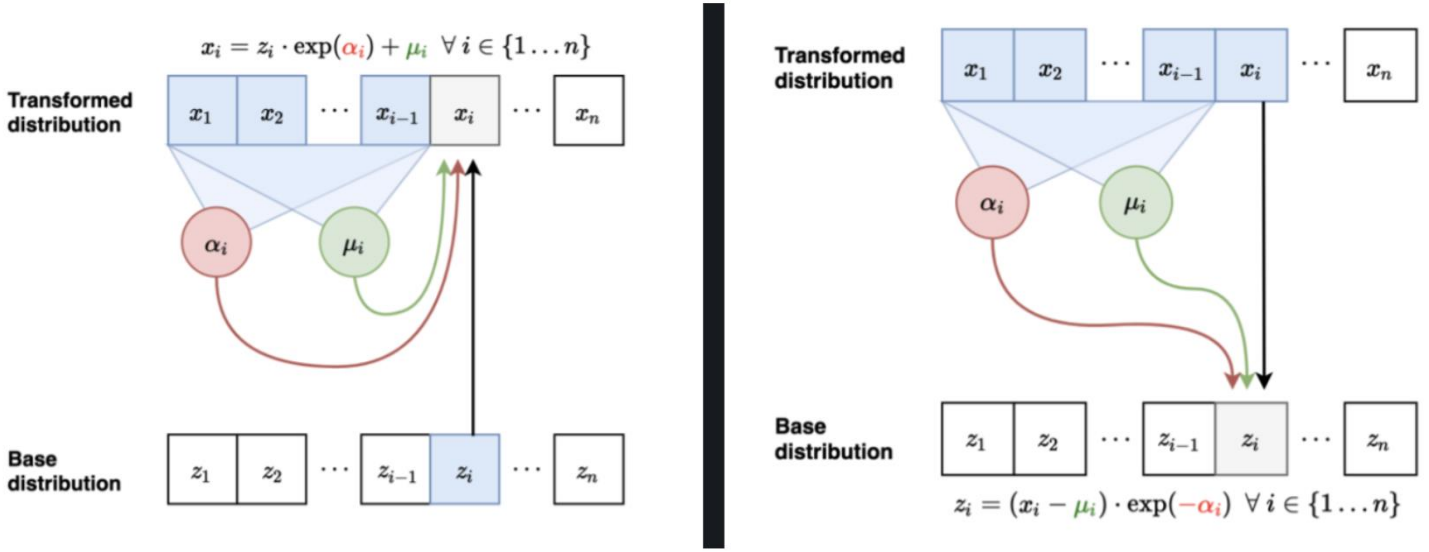


Fig 4 : Forward pass of MAFs (left) vs. Inverse pass of MAFs (right).

- Sampler for this model:
  - Sample $z_i \sim \mathcal{N}(0,1)$ for $i = 1, \cdots, n$
  - Let $x_1 = \exp(\alpha_1) z_1 + \mu_1$. Compute $\mu_2(x_1), \alpha_2(x_1)$
  - Let $x_2 = \exp(\alpha_2) z_2 + \mu_2$. Compute $\mu_3(x_1, x_2), \alpha_3(x_1, x_2)$
  - Let $x_3 = \exp(\alpha_3) z_3 + \mu_3$. ...
- **Flow interpretation:** transforms samples from the standard Gaussian $(z_1, z_2, \dots, z_n)$ to those generated from the model $(x_1, x_2, \dots, x_n)$ via invertible transformations (parameterized by $\mu_i(\cdot), \alpha_i(\cdot)$)
  - Forward mapping from $\mathbf{z} \mapsto \mathbf{x}$:
    - Let $x_1 = \exp(\alpha_1) z_1 + \mu_1$. Compute $\mu_2(x_1), \alpha_2(x_1)$
    - Let $x_2 = \exp(\alpha_2) z_2 + \mu_2$. Compute $\mu_3(x_1, x_2), \alpha_3(x_1, x_2)$
  - Sampling is sequential and slow (like autoregressive): $O(n)$ time
- Inverse mapping from $\mathbf{x} \mapsto \mathbf{z}$:
  - Compute **all** μ_i, α_i (can be done in parallel using e.g., MADE)
  - Let $z_1 = (x_1 - \mu_1) / \exp(\alpha_1)$ (scale and shift)
  - Let $z_2 = (x_2 - \mu_2) / \exp(\alpha_2)$
  - Let $z_3 = (x_3 - \mu_3) / \exp(\alpha_3)$ ...
- Jacobian is lower diagonal, hence efficient determinant computation
- Likelihood evaluation is easy and parallelizable (like MADE)
- Layers with different variable orderings can be stacked

Papamakarios et al., Masked Autoregressive Flow for Density Estimation (arXiv:1705.07057)

Inverse Autoregressive Flows (IAFs)

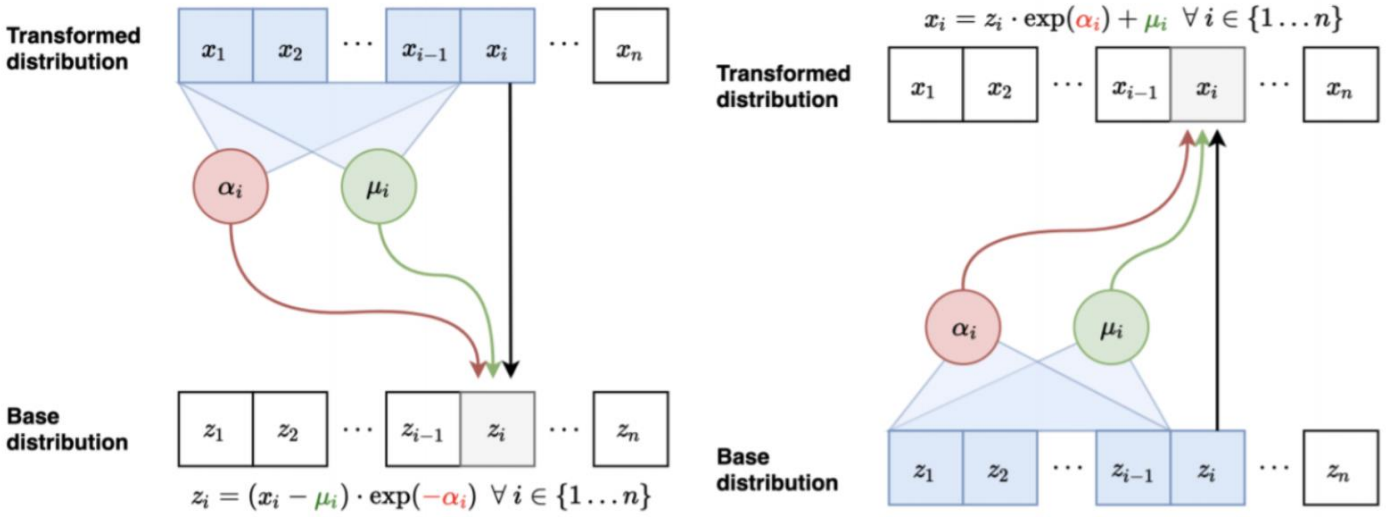


Fig 5 : Inverse pass of MAF (left) vs. Forward pass of IAF (right).

- Forward mapping from $\mathbf{z} \mapsto \mathbf{x}$ (parallel):
  - Sample $z_i \sim \mathcal{N}(0, 1)$ for $i = 1, \cdots, n$
  - Compute all μ_i, α_i (can be done in parallel)
  - Let $x_1 = \exp(\alpha_1) z_1 + \mu_1$
  - Let $x_2 = \exp(\alpha_2) z_2 + \mu_2$ ...
- Inverse mapping from $\mathbf{x} \mapsto \mathbf{z}$ (sequential):
  - Let $z_1 = (x_1 - \mu_1) / \exp(\alpha_1)$. Compute $\mu_2(z_1), \alpha_2(z_1)$
  - Let $z_2 = (x_2 - \mu_2) / \exp(\alpha_2)$. Compute $\mu_3(z_1, z_2), \alpha_3(z_1, z_2)$

Kingma et al., Improved Variational Inference with Inverse Autoregressive Flow (arXiv:1606.04934)

MAF vs IAF

MAFs can compute the density $p(x)$ very efficiently. During the training process, all the $x_i's$ are known. One can therefore parallelize this step using masks. Sampling on the other hand (predicting x_D) is slower as it requires it requires performing D sequential passes (where D is the dimensionality of x) to calculate the previous samples $(x_1, x_2 \ldots x_{D-1})$ before computing x_D.

IAF can generate samples efficiently with one pass through the model since all the $z_i's$ are known. However, the training process is slow, since estimating the z_i requires i sequential passes to calculate all the required input variables z_1, z_2, .. z_{i-1}.

Plain English: MAF = fast density evaluation (good for training), slow sampling. IAF = the reverse. Choose based on if you need fast training or fast generation.

MAF vs IAF, *Continued*

- Computational tradeoffs
  - MAF: Fast likelihood evaluation, slow sampling
  - IAF: Fast sampling, slow likelihood evaluation
- MAF more suited for training based on MLE, density estimation
- IAF more suited for real-time generation
- Can we get the best of both worlds?
  - Two part training with a teacher and student model
  - Teacher is parameterized by MAF. Teacher can be efficiently trained via MLE
  - Once teacher is trained, initialize a student model parameterized by IAF. Student model cannot efficiently evaluate density for external datapoints but allows for efficient sampling
  - **Key observation**: IAF can also efficiently evaluate densities of its own generations (via caching the noise variates $z_1, z_2, \ldots, z_n$)

Masked Auto-Encoder for Distribution Estimation (MADE)

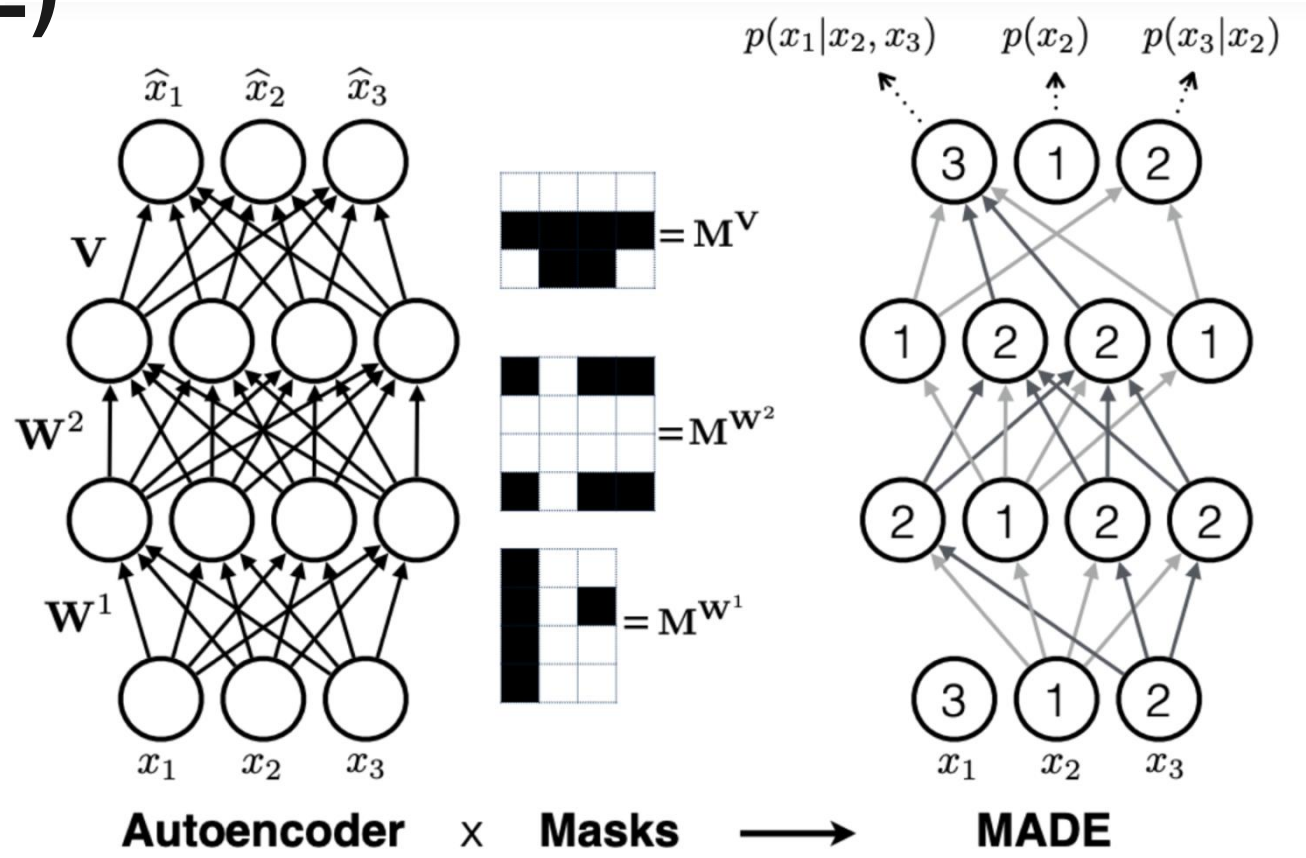


Fig 3 : This image is taken from the MADE paper an explains the idea of masking, so as to parallelizes the sequential autoregressive computation.

Germain et al., MADE: Masked Autoencoder for Distribution Estimation (arXiv:1502.03509)